Bank Interest-Rate Sensitivity

Reproducible Empirical Analysis of U.S. Commercial Banks, 1984–2017

Cunhan Guo

Reproducibility. The scripts in `code/` read the four raw files under `data/` and write processed data, tables, and figures under `data_processed/` and `output/`. Run scripts 01 through 06 in order from the project root. The bank-quarter panel spans 1984Q1–2017Q4.

1 Data Audit and Descriptive Evidence

1.1 Data and data-preparation choices

Four files are provided. `callreports_1984_2017.dta` holds 1,114,194 bank-quarter rows for 17,804 distinct commercial-bank identifiers, calendar quarter, quarterly average assets, total interest income, total interest expense, and net income. After excluding rows with missing flows or non-positive assets, 17,743 banks remain with at least one usable quarter; 61 raw identifiers have no usable observations. `FedFunds.csv` is the monthly effective federal funds rate; I take the last month of each quarter as f_t. `Bank_and_CRSP_Linking_Table.csv` and `CRSP_Monthly_Stock_File.csv` are used in Section 5.

The three income/expense/net-income items are year-to-date (YTD) cumulative flows, not single-quarter flows. I detected this because ROA built from raw net income had a seasonal sawtooth: Q4 values ran systematically above Q1 for almost every bank. I verified the cumulation directly: in complete bank-years, the reconstructed quarterly flows sum exactly to the reported Q4 YTD value. Single-quarter flows are recovered by within-year differencing (Q1 flow = Q1 YTD; quarters 2–4: flow = YTD_t - YTD_{t-1}). One timing guard: I difference only across adjacent calendar quarters (using the year and quarter fields). A bank that skips a quarter gets a missing flow for that quarter rather than a two-quarter change mislabeled as one quarter. Skipped quarters are rare but concentrated among short-lived banks.

Missing values: 5,676 of 1,114,194 rows are dropped for missing flows or non-positive assets (0.51%), leaving 1,108,518 bank-quarters. Missingness is highest in the mid-1980s and around 2005–06 and is near zero after 2010 (Figure 2). Extreme observations: the three annualized rates are winsorized at the 0.5/99.5 percentiles within each calendar quarter before beta estimation, so a single merger or write-down quarter does not drive one bank's time-series regression. The estimated betas are not trimmed further.

Fixed effects follow the variation used in each regression. The bank-level time-series models include bank-specific intercepts; quarter effects would absorb the common federal-funds-rate changes used to estimate the betas. The cross-sections have one observation per bank, so bank fixed effects cannot be estimated alongside the bank-level beta outcome; I use HC1 standard errors. In the company-month stock-return regressions, company fixed effects would change the question from between-company associations to within-company changes. Month effects would absorb the common market return, so I include the CRSP market return and cluster standard errors by company.

Table 1: Descriptive statistics, bank-quarter panel, 1984–2017 (N = 1,108,518 bank-quarters, 17,743 banks). Assets are reported by the Call Report in $ thousands and converted to $ millions. Rates are annualized percent. Rates are winsorized at 0.5/99.5 within quarter; the _w suffix is dropped in the text and tables.

Variable	N	Mean	SD	p1	Median	p99
Ave assets ($m)	1108518	757.438	20333.837	5.716	66.104	6139.083
Total assets ($m)	1108518	765.824	20505.106	5.794	66.840	6251.376
IncRate (% p.a.)	1108518	7.441	2.262	2.776	7.541	12.482
ExpRate (% p.a.)	1108518	3.406	1.917	0.142	3.344	7.545
ROA (% p.a.)	1108518	0.823	1.597	-6.022	1.038	3.803
Fed funds rate (%)	1108518	4.737	3.019	0.080	5.290	11.300

The median bank is small (median average assets $66.1m versus a mean of $757m), so a few hundred large banks account for most industry assets. Median IncRate is 7.54%, median ExpRate 3.34%, and median ROA 1.04%, with the average federal funds rate over the sample at 4.74%. Table 1 shows the full distribution.

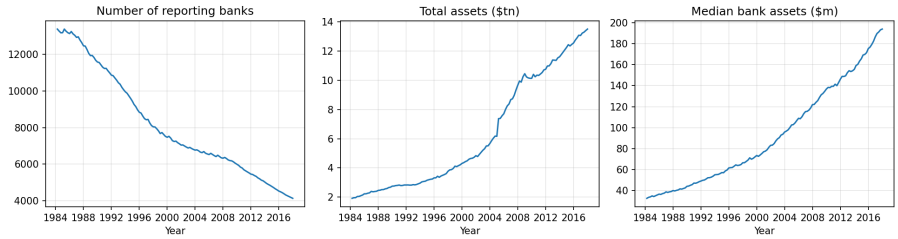


Figure 1: Banking industry over time: number of reporting banks (left), total assets (center), median bank assets (right).

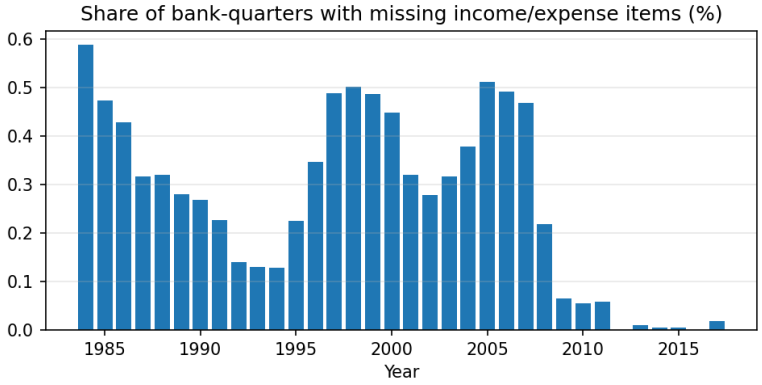


Figure 2: Share of bank-quarters with missing income/expense/net-income items by year, computed on the raw panel before dropping rows.

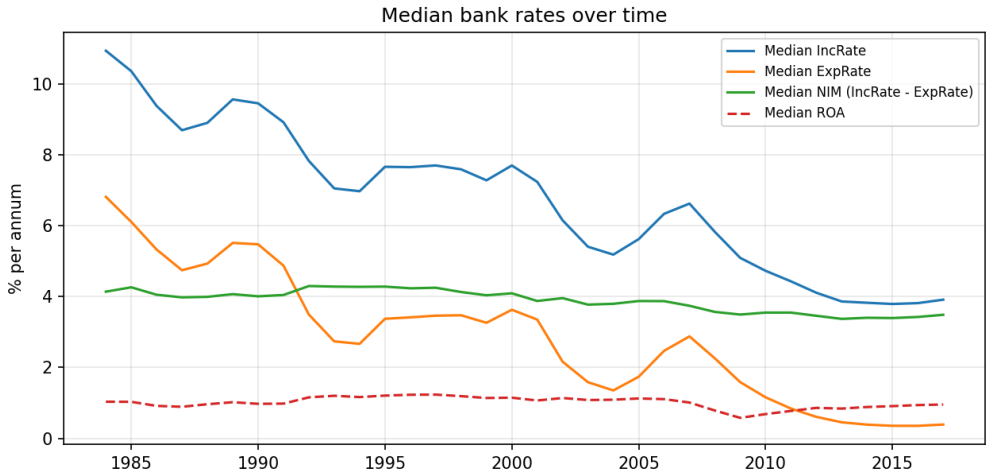


Figure 3: Annual median IncRate, ExpRate, net interest margin (IncRate minus ExpRate), and median ROA, 1984–2017. The margin is remarkably stable near 4 percentage points; ROA dips in 1987, 1991, and 2008–09.

1.2 Main patterns

Three patterns stand out. First, the industry consolidated sharply: reporting banks fell from about 13,500 in 1984 to about 4,500 by 2016, while total assets grew more than sixfold (Figure 1). Second, the size distribution stayed extremely skewed (mean assets \$757m, median \$66m). Third, the net interest margin hovered near 4 percentage points for the whole sample, while median ROA moved with the cycle (Figure 3).

Two data limitations matter for what follows. The panel is unbalanced and exit is correlated with size, so time-series aggregates mix entry, exit, and organic change. And the YTD reporting convention, if left untreated, would put a mechanical seasonal component into every rate and every beta; the reconstruction in the previous subsection is the main guard against that.

2 Bank-Level Interest Income and Expense Betas

For bank i in quarter t I construct annualized rates in percentage points:

$$\begin{aligned} \text{IncRate}_{i,t} &= 400 \times \frac{\text{interest income}_{i,t}}{\text{average assets}_{i,t}}, \\ \text{ExpRate}_{i,t} &= 400 \times \frac{\text{interest expense}_{i,t}}{\text{average assets}_{i,t}}. \end{aligned} \tag{1}$$

using the reconstructed single-quarter flows. The federal funds rate f_t is the last monthly value of quarter t. First differences are

$$\Delta y_{i,t} = y_{i,t} - y_{i,t-1}, \qquad \Delta f_t = f_t - f_{t-1}, \tag{2}$$

and, for each bank and each outcome $y \in \{\text{IncRate}, \text{ExpRate}, \text{ROA}\}$, I estimate

$$\Delta y_{i,t} = a_i^y + \sum_{\tau=0}^{3} b_{i,\tau}^y \, \Delta f_{t-\tau} + u_{i,t}^y, \tag{3}$$

with the four coefficients allowing the response to build over one year. The bank's beta is the sum of the four estimated coefficients,

$$\widehat{\beta}_i^y = \sum_{\tau=0}^{3} \widehat{b}_{i,\tau}^y. \tag{4}$$

I require at least 20 usable first-difference observations per bank (about five years). This gives 13,624 eligible banks of 17,743 (77%); most excluded banks are short-lived or de novo. Median one-year betas are 0.358 for income and 0.353 for expense. A one-percentage-point rise in the federal funds rate is associated, within a year, with about a 0.36-point rise in the income rate and a 0.35-point rise in the expense rate. The ROA beta distribution is centered near zero (median -0.012, SD 0.70), so profitability is only loosely tied to rate movements on average (Table 2, Figure 4).

Table 2: Distribution of bank-level estimated betas (13,624 banks). Rates are winsorized at 0.5/99.5 within quarter; the _w suffix is dropped in the text and tables. Betas are not trimmed further.

Statistic	Beta_Inc	Beta_Exp	Beta_ROA
N banks	13624	13624	13624
Mean	0.369	0.354	-0.014
SD	0.276	0.145	0.703
p10	0.183	0.230	-0.360
p25	0.269	0.294	-0.134
Median	0.358	0.353	-0.012
p75	0.464	0.418	0.114
p90	0.594	0.492	0.363

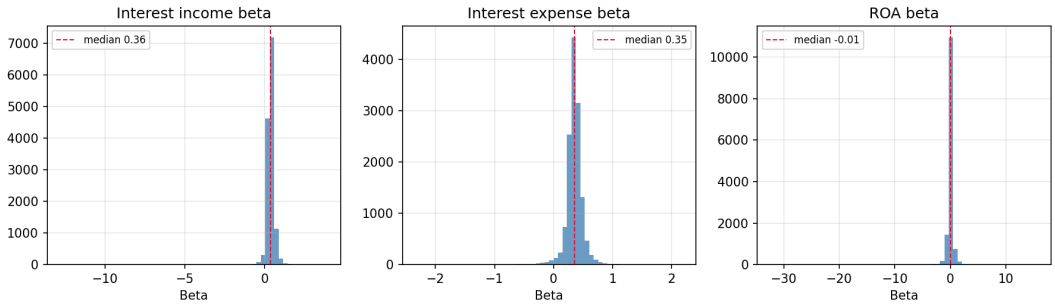


Figure 4: Histograms of bank-level income, expense, and ROA betas. The ROA distribution is much wider than the two rate betas.

The cleaned bank-level dataset is submitted as `data_processed/bank_betas.csv`: one row per bank (17,743 rows) with idrssd, bank name, first/last quarter, mean assets, the three betas (beta_Inc, beta_Exp, beta_ROA), observation counts, R^2, and eligibility flags. Banks below the 20-observation minimum keep a row with missing betas, so the file contains every bank in the panel. Size in that file is each bank's time-series average of total assets over its usable sample (mean_asset), the natural one-row-per-bank analogue of the quarterly asset measure.

3 Relationship between Income and Expense Sensitivities

I regress income beta on expense beta by OLS:

$$\widehat{\beta}_i^{\text{Inc}} = \alpha + \gamma\, \widehat{\beta}_i^{\text{Exp}} + \varepsilon_i. \tag{5}$$

I use HC1 heteroskedasticity-robust standard errors for the cross-sectional regressions. The estimates are based on one observation per bank; the table reports coefficients, HC1 t-statistics, sample sizes, and adjusted R-squared values.

Table 3: Cross-sectional regressions of income beta on expense beta (Panel A) and ROA beta on expense beta (Panel B), one observation per bank. HC1 t-statistics in parentheses. Size terciles are defined by each bank's time-series average total assets (mean_asset), not within-quarter assets. For interaction rows, 'Slope' is the expense-beta coefficient at average log assets; the level term in log assets is not shown. Stars denote two-sided significance at 10%, 5%, and 1% (*, **, ***).

Specification	Intercept	Slope	N	Adj R2	Interaction
A: Inc on Exp (full)	0.116*** (3.76)	0.715*** (8.17)	13624	0.14	
B: ROA on Exp (full)	-0.257** (-2.11)	0.688** (2.03)	13624	0.02	
A: Inc on Exp, Small	0.174*** (13.53)	0.529*** (14.16)	4541	0.117	
A: Inc on Exp, Medium	0.087** (2.08)	0.797*** (6.92)	4541	0.213	
A: Inc on Exp, Large	0.097 (1.25)	0.777*** (3.76)	4542	0.114	
B: ROA on Exp, Small	-0.092*** (-2.59)	0.165* (1.66)	4541	0.001	
B: ROA on Exp, Medium	-0.478 (-1.44)	1.323 (1.43)	4541	0.05	
B: ROA on Exp, Large	-0.188 (-1.48)	0.521 (1.53)	4542	0.013	
A: interaction with log assets	0.115*** (3.84)	0.707*** (9.45)	13624	0.147	0.092 (0.76)
B: interaction with log assets	-0.258** (-2.06)	0.687** (2.01)	13624	0.02	0.027 (0.16)

The estimated slope is positive and below one (Table 3, Panel A): a 0.1 higher expense beta corresponds to about a 0.072 higher income beta. This is consistent with asset and liability repricing moving together, while income rates respond less than one-for-one on average. The positive intercept indicates that banks with low expense sensitivity can still have positive income sensitivity, as would be expected when some assets reprice with short-term rates.

The relationship is positive in each size tercile, with a somewhat larger slope among medium and large banks than among small banks. The interaction between expense beta and demeaned log assets is not statistically significant; the data do not show a clear linear size gradient (Figure 5).

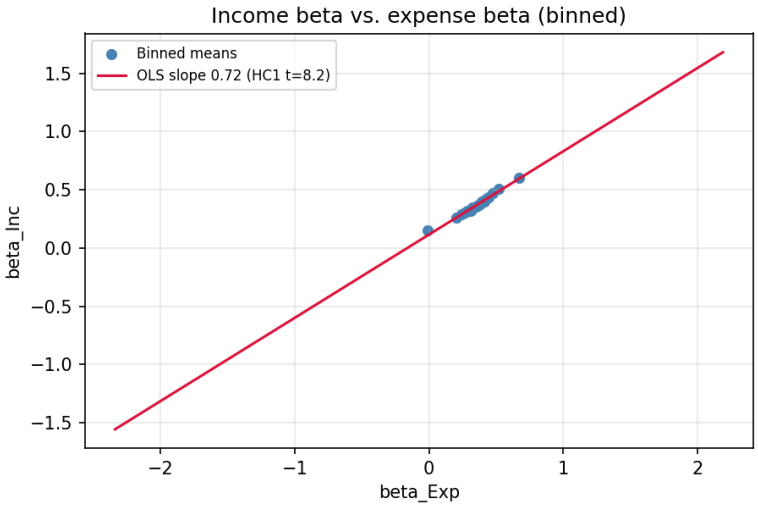


Figure 5: Binned scatter of income beta against expense beta (20 equal-frequency bins) with the OLS line. All eligible banks are included.

4 Interest Sensitivity of Profitability

ROA is net income divided by average assets, annualized:

$$\text{ROA}_{i,t} = 400 \times \frac{\text{quarterly net income}_{i,t}}{\text{quarterly average assets}_{i,t}}, \tag{6}$$

and the ROA beta comes from the same distributed-lag regression. Regressing beta_ROA on beta_Exp gives a positive slope (HC1 t = 2.03), with adjusted $R^2 = 0.020$ (Table 3, Panel B). Expense beta explains only a small share of cross-bank variation in ROA beta. The negative intercept is hard to interpret mechanically because very few banks have exactly zero expense sensitivity.

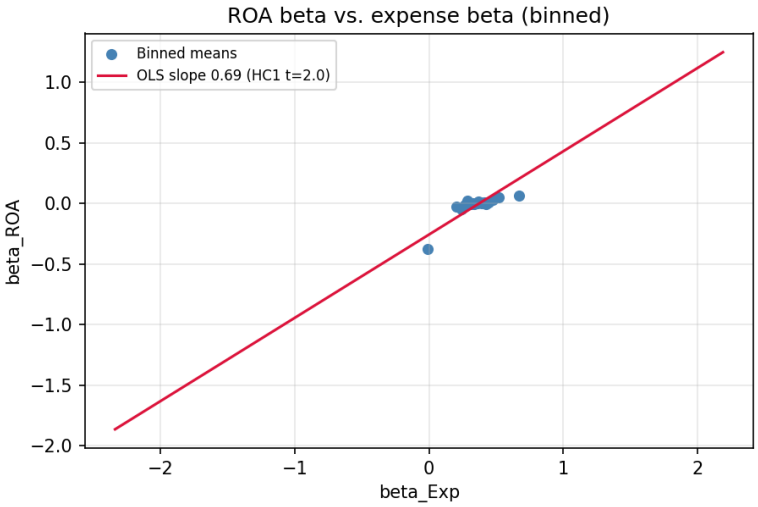


Figure 6: Binned scatter of ROA beta against expense beta with the OLS line. The positive slope is driven mostly by the upper bins; the fit is weak everywhere ($R^2 = 0.02$).

Size heterogeneity: the estimated slopes are positive across size groups, but none is statistically distinguishable from zero at conventional levels. The interaction with demeaned log assets is also insignificant (Figure 6). The overall message of this section is that profitability sensitivity is weakly related to expense sensitivity, with most cross-bank variation left unexplained.

5 Publicly Traded Banks

5.1 Linking banks to stocks

I map the bank panel to CRSP with `Bank_and_CRSP_Linking_Table.csv`, which links RSSD identifiers (bank idrssd, holding-company rssd9001) to PERMCO. Matching is on the bank's own idrssd first; banks with no idrssd match are then tried through the holding company's bhc_id. I chose this order after checking that a naive concat of the two match types could put the same bank under two different PERMCOs and double-count its assets and flows in the company aggregates. Each link is restricted to its validity window [dt_start, dt_end]. This yields 124,929 bank-quarter links covering 5,163 banks and 1,062 listed companies (PERMCO). No bank-quarter has multiple PERMCOs among the valid links.

Where several banks map to one company, I aggregate to the company level: average assets, total assets, and the three quarterly flows are summed (stocks and flows both aggregate linearly), and the rates are recomputed from the sums. These records include only banks matched to a listed company, so company assets and flows may be understated when subsidiaries are unmatched; ambiguous links are excluded. Intra-company transfers are not netted out, and partly owned subsidiaries are counted at 100% rather than by ownership share. Call Report data do not let me separate these components, so the resulting consolidated rates may be biased.

5.2 Total-return index

CRSP lists multiple share classes (PERMNO) for the same company-month. Before building any series I collapsed them using prior-month market-cap weights, so a company's monthly return is the value-weighted average of its share classes. The CRSP file contains 453,756 PERMCO-month duplicates from multiple share classes. Monthly returns use MthRet, which includes dividends, so the compounded series is a total-return index (2000 = 100).

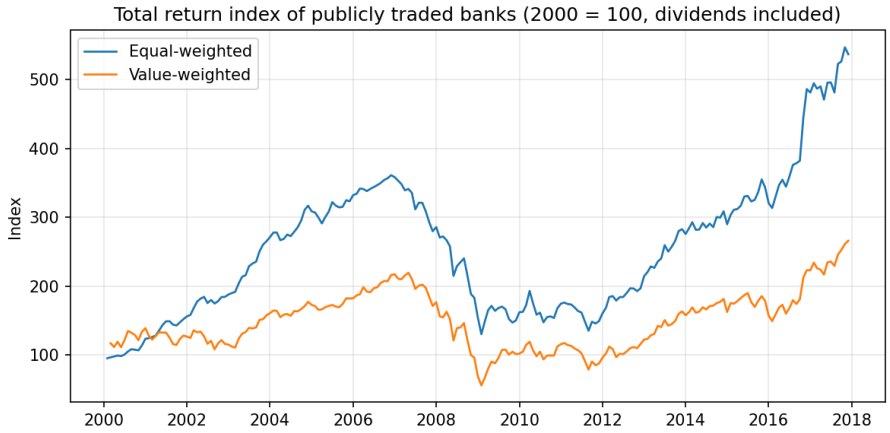


Figure 7: Total return index of publicly traded banks (2000 = 100, dividends included), equal-weighted and value-weighted, 2000–2017. The value-weighted index lost about 60% peak-to-trough in 2008–09; the equal-weighted index fell far less, so most crisis losses were concentrated in the big banks.

Figure 7 shows that the value-weighted index fell much more during 2008–09 and recovered more slowly than the equal-weighted index. Returns among the largest listed bank companies drove much of the value-weighted series' decline.

5.3 Bank characteristics and stock returns

To avoid look-ahead bias, each quarter's Call Report is treated as public two months after quarter end (reports are filed with a lag); each monthly return is matched to the most recent report whose public date is no later than that month. The company-level expense beta is not used in return regressions because a full-sample beta would embed post-return information; return tests use only lagged accounting levels. After aggregation, company-level IncRate, ExpRate, and ROA are recomputed from summed flows and assets, then winsorized at the 0.5/99.5 percentiles within quarter. This applies the same percentile cutoffs as the bank-level rates while defining outliers across linked companies.

Table 4: Monthly stock returns of listed banks on lagged company characteristics, 2000–2017. Coefficient (t-statistic clustered by listed company). vwretd is the CRSP value-weighted market return; company rates are winsorized within quarter. Stars denote two-sided significance at 10%, 5%, and 1% (*, **, ***).

Variable	(1) Market	(2) +Size+ROA	(3) +Rates
Constant	0.0053*** (12.54)	0.0123*** (3.62)	0.0023 (0.57)
Market return (vwretd)	0.6137*** (36.62)	0.6169*** (36.87)	0.6148*** (36.69)
Log total assets		-0.0006*** (-2.80)	-0.0005** (-2.43)
ROA (%)		0.0024*** (4.10)	0.0020*** (3.64)
Income rate (%)			0.0030*** (5.33)
Expense rate (%)			-0.0041*** (-5.27)
N	77720	77720	77720
R-squared	0.0592	0.0606	0.0612

The market-return coefficient is positive and precisely estimated. Conditional on the market return, larger company-level assets are associated with slightly lower subsequent monthly returns. In the rate specification, income rate enters positively and expense rate negatively, consistent with

a positive association between net interest margins and subsequent returns (Table 4). All three specifications use the same 77,720 company-month observations.

6 Is the Pass-Through of Policy Rates to Bank Funding Costs Symmetric?

6.1 Empirical approach

Section 2 estimated a single expense beta, implicitly assuming that funding costs respond symmetrically to rate increases and decreases. Deposit rates are often described as sticky downward: banks reprice deposits quickly when the Fed raises, but cut them slowly when the Fed lowers. If so, a cut in the policy rate should pass through to funding costs less than a hike of the same size, widening margins during easing cycles. I test this directly by splitting each quarterly change in the federal funds rate into an increase and a decrease part,

$$\Delta f_t^+ = \max(\Delta f_t, 0), \qquad \Delta f_t^- = \min(\Delta f_t, 0), \tag{7}$$

and estimating, for each bank,

$$\Delta \text{ExpRate}_{i,t} = a_i + \sum_{\tau=0}^{3} b_{i,\tau}^{\text{up}} \, \Delta f_{t-\tau}^+ + \sum_{\tau=0}^{3} b_{i,\tau}^{\text{down}} \, \Delta f_{t-\tau}^- + u_{i,t}, \tag{8}$$

with the two cumulative betas

$$\beta_i^{\text{up}} = \sum_{\tau=0}^{3} \widehat{b}_{i,\tau}^{\text{up}}, \qquad \beta_i^{\text{down}} = \sum_{\tau=0}^{3} \widehat{b}_{i,\tau}^{\text{down}}. \tag{9}$$

The identification compares how a bank's expense rate moves in quarters following rate hikes versus quarters following rate cuts, over the same distributed-lag window as in Section 2. I require at least 28 usable observations (seven years), leaving at least 19 residual degrees of freedom after estimating eight lag coefficients and an intercept. The table reports the eligible sample sizes. Size terciles use each bank's time-series mean of total assets, as in Section 3. Estimated betas are not trimmed. All results are descriptive, not causal.

Table 5: Asymmetry in interest-expense pass-through. Mean difference is $\beta_{up} - \beta_{down}$; positive values mean greater pass-through during hikes. Share reports the fraction of banks with a positive difference.

Sample	N	Mean up	Mean down	Mean diff.	Median diff.	Share with diff > 0	Paired t	p-value
Full sample (1984-2017)	12242	0.385	0.355	+0.029	+0.006	51.2%	7.61	<0.001
Small	4081	0.322	0.352	-0.030	-0.026	43.6%	-5.23	<0.001
Medium	4080	0.386	0.351	+0.035	+0.004	50.8%	5.14	<0.001
Large	4081	0.445	0.364	+0.082	+0.036	59.3%	11.70	<0.001
Pre-2007 (re-estimated)	11403	0.352	0.402	-0.050	-0.060	35.6%	-17.17	<0.001

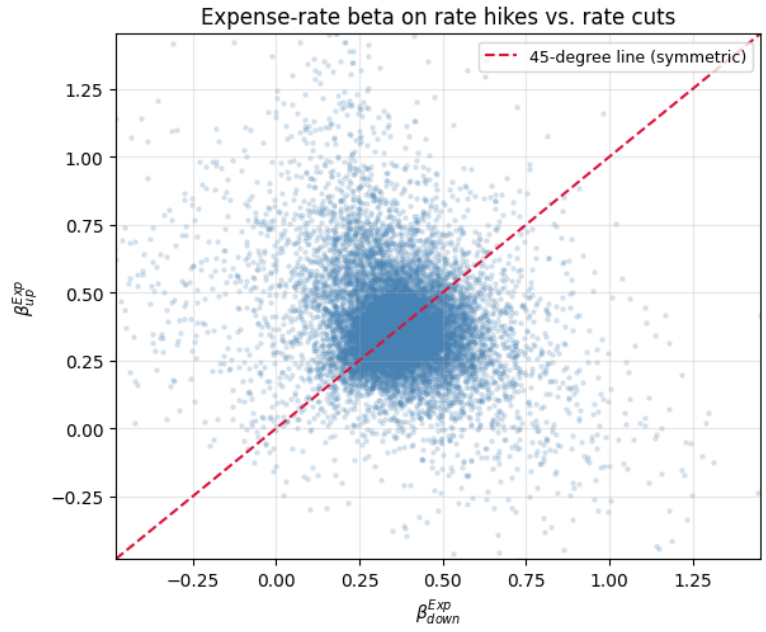


Figure 8: Bank-level cumulative expense-rate betas for rate hikes and cuts. The dashed line shows equal pass-through; axes show the central 99% of estimates. All eligible banks enter the summary table.

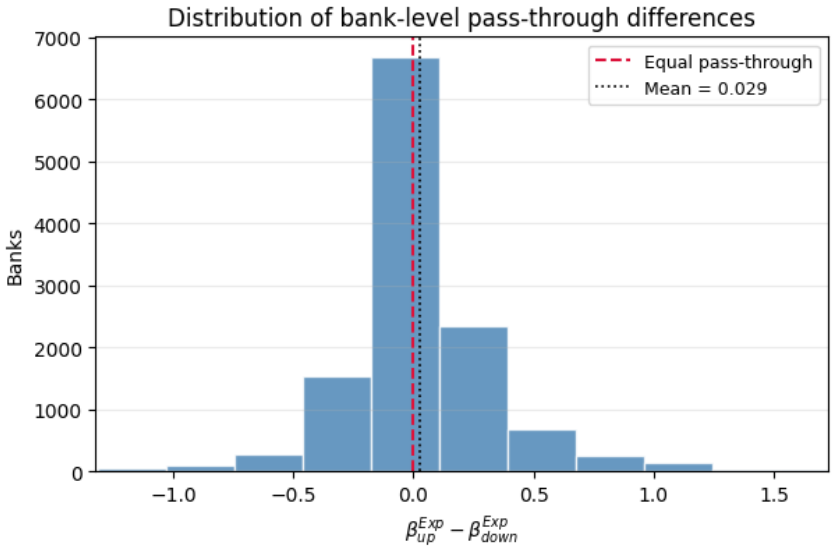


Figure 9: Distribution of full-sample bank-level differences, $\beta_{up} - \beta_{down}$. Axes show the central 99% of estimates; all banks are included in the table. The dashed line marks zero and the dotted line marks the sample mean.

6.2 Results

Table 5 reports the mean hike-minus-cut difference with its sign shown explicitly. In the full sample it is +0.029; the pre-2007 re-estimate is -0.050. The sign reversal means the full-sample result does not show a stable asymmetry across periods. Figure 8 compares the two betas directly, while

Figure 9 shows the distribution of their differences.

By size, mean differences are -0.030, +0.035, and +0.082 for small, medium, and large banks, respectively. These descriptive tercile comparisons do not establish a causal size pattern.

The sign has a direct interpretation: if $\beta_{up} > \beta_{down}$, expense rates pass through more during hikes, consistent with slower adjustment during cuts; if the two betas are equal, pass-through is symmetric; if $\beta_{down} > \beta_{up}$, pass-through is greater during cuts. Moreover, aggregate interest expense includes more than deposit costs, so this measure does not identify deposit pricing. Nor does the comparison by itself establish how net interest margins move, because asset yields also reprice. Hikes and cuts differ in number and size, bank-level betas are estimated with noise, and the results apply to banks meeting the 28-observation minimum. These estimates are descriptive, not causal.

7 Conclusion

Income and expense rates respond positively to federal funds rate changes, and their bank-level betas move together. Profitability betas are only weakly related to expense betas. In the stock-return regressions, the market return is strongly associated with bank stock returns; accounting-rate associations are modest. The hike-minus-cut difference is positive in the full sample but negative in the pre-2007 re-estimation, so the evidence does not support a stable asymmetry across periods. All findings are descriptive, not causal.

A Reproducible Code

All seven scripts are released with this report. They read the four raw files (Call Report, federal-funds series, bank–CRSP link table, CRSP monthly stock file) from `data/`, write processed data to `data_processed/`, and write tables and figures to `output/`; run them in the order shown. The code is reproduced here verbatim, with line numbers.

Listing 1: `01_data_prep.py` — Read the raw Call Report, undo YTD cumulation, annualize and winsorize the rates, and build the bank-quarter panel, descriptive table, and data-audit figures.

```
"""
Bank Interest-Rate Sensitivity
Script 01: Data preparation from raw files

Inputs (raw, unmodified):
  - data/callreports_1984_2017.dta
  - data/FedFunds.csv
Outputs:
  - data_processed/bank_panel.parquet
  - output/tables/table1_descriptives.csv
  - output/figures/fig1_industry_over_time.png
  - output/figures/fig2_missing_values.png
  - output/figures/fig3_rate_trends.png

Things I hit while writing this, kept so the reader sees the reasoning:

  * The income/expense/net-income items are year-to-date (YTD) cumulative
    flows. I first built ROA directly from reported "net income" and got a
    seasonal sawtooth: Q4 ROA was systematically higher than Q1 ROA for nearly
    every bank. That is the tell for a YTD variable. I then checked complete
    bank-years: the four reconstructed quarterly flows equal reported Q4 YTD.
  * My first version differenced YTD across any same-year rows, which turned a
    skipped Q2 into a Q3 flow of Q2+Q3. Now I difference only adjacent quarters
    and set skipped quarters to missing.
  * ave_asset is a quarterly-average stock, not a flow, so it is NOT differenced.
  * Missingness is small but non-random over time (more in the 1980s and again
    around 2005-06); I drop those rows rather than impute.
  * I winsorize rates at 0.5/99.5 within quarter before beta estimation so a
    handful of merger quarters do not drive an individual bank's regression.
"""
import numpy as np
import pandas as pd
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
DATA = ROOT / "data"
PROC = ROOT / "data_processed"
OUT_TAB = ROOT / "output" / "tables"
OUT_FIG = ROOT / "output" / "figures"
for p in [PROC, OUT_TAB, OUT_FIG]:
    p.mkdir(parents=True, exist_ok=True)

# ---------------------------------------------------------------- load
# Read the raw .dta directly (no cached parquet), keeping only the columns
# used. This guarantees the script runs from the provided raw file.
cols = ["idrssd", "qdate", "year", "quarter", "charter_type", "bhc_id",
        "bank_name", "total_asset", "ave_asset", "interest_income",
        "interest_expense", "net_income"]
```

```python
df = pd.read_stata(f"{DATA}/callreports_1984_2017.dta", columns=cols, convert_categoricals=False)
df = df.sort_values(["idrssd", "qdate"]).reset_index(drop=True)
df["qindex"] = df["year"].astype(int) * 4 + df["quarter"].astype(int) - 1

# calendar quarter-end date (last day of the quarter); used for the Fed funds
# merge and for the CRSP "public from" timing in script 04.
_qm = {1: 3, 2: 6, 3: 9, 4: 12}
_qd = {1: 31, 2: 30, 3: 30, 4: 31}
df["qdate_dt"] = pd.to_datetime(
    df["year"].astype(str) + "-" + df["quarter"].map(_qm).astype(str).str.zfill(2)
    + "-" + df["quarter"].map(_qd).astype(str).str.zfill(2)
)

# --- restore single-quarter flows from YTD items ---------------------
# qindex is a consecutive-quarter integer built from year and quarter. Only
# difference when the previous row for the same bank is the immediately
# preceding calendar quarter (qindex diff == 1). A skipped quarter is set to
# missing, not faked by differencing across the gap.
dq = df.groupby("idrssd")["qindex"].diff()
adjacent = dq == 1
for v in ["interest_income", "interest_expense", "net_income"]:
    prev = df.groupby("idrssd")[v].shift(1)
    is_q1 = df["quarter"] == 1
    df[v + "_q"] = np.where(is_q1, df[v],
                            np.where(adjacent & prev.notna(), df[v] - prev, np.nan))

# Validate the reconstruction only for complete bank-years with all four
# quarters observed and reconstructed. Incomplete years cannot be compared
# with a Q4 YTD total.
annual = df.groupby(["idrssd", "year"]).agg(
    quarters=("quarter", lambda s: tuple(sorted(s.dropna().unique()))),
    n_rows=("quarter", "size"),
)
complete_keys = annual.index[annual["quarters"].map(lambda x: x == (1, 2, 3, 4)) & annual["n_rows"
    ].eq(4)]
for v in ["interest_income", "interest_expense", "net_income"]:
    qsum = df.groupby(["idrssd", "year"])[v + "_q"].sum(min_count=4)
    q4 = df.loc[df["quarter"].eq(4)].set_index(["idrssd", "year"])[v]
    chk = pd.concat([qsum.rename("qsum"), q4.rename("q4_ytd")], axis=1).loc[complete_keys].dropna()
    err = (chk["qsum"] - chk["q4_ytd"]).abs()
    print(f"[check] {v}: complete bank-years={len(chk)}, exact within 1e-6={(err < 1e-6).mean()
    :.3%}, max abs diff={err.max():.6g}")

# ------------------------------------------------------------- rates
# filter first (non-positive or missing asset / flow rows) so the rate
# denominators are strictly positive before dividing.
before = len(df)
raw_banks = df["idrssd"].nunique()
df = df.dropna(subset=["ave_asset", "interest_income_q", "interest_expense_q", "net_income_q"])
df = df[(df["ave_asset"] > 0) & (df["total_asset"] > 0)]
print(f"[drop] rows {before} -> {len(df)}")

df["IncRate"] = 400.0 * df["interest_income_q"] / df["ave_asset"]   # % per year
df["ExpRate"] = 400.0 * df["interest_expense_q"] / df["ave_asset"]
df["ROA"]     = 400.0 * df["net_income_q"] / df["ave_asset"]

# ------------------------------------------------------- Fed funds
ff = pd.read_csv(f"{DATA}/FedFunds.csv")
ff["date"] = pd.to_datetime(ff["date"])
```

```
ff = ff.sort_values("date").reset_index(drop=True)
# effective fed funds rate at the end (last month) of each calendar quarter
ff["qend"] = ff["date"].dt.to_period("Q").dt.end_time.dt.normalize()
qff = ff.groupby("qend")["FedFunds"].last().reset_index()
qff.columns = ["qdate_dt", "ffrate"]
df = df.merge(qff, on="qdate_dt", how="left")

# ------------------------------------------------------------------ FE
df["log_asset"] = np.log(df["total_asset"])

# winsorized versions of the rates (0.5/99.5, cross-section by quarter).
# The _w suffix is internal; the report/tables drop it.
def winsor(s, lo=0.005, hi=0.995):
    l, h = s.quantile(lo), s.quantile(hi)
    return s.clip(l, h)

df["IncRate_w"] = df.groupby("qdate")["IncRate"].transform(lambda s: winsor(s))
df["ExpRate_w"] = df.groupby("qdate")["ExpRate"].transform(lambda s: winsor(s))
df["ROA_w"] = df.groupby("qdate")["ROA"].transform(lambda s: winsor(s))

# ------------------------------------------------------------------ Table 1
# Assets are reported in $ thousands; I convert to $ millions in the table so
# the magnitudes are readable (median bank ~$66m of assets). Only the main
# variables used downstream are shown (the winsorized rates are the ones used
# in estimation; the raw versions are not reported separately).
tab = pd.DataFrame(
    {
        "Variable": [
            "Ave assets ($m)", "Total assets ($m)", "IncRate (% p.a.)",
            "ExpRate (% p.a.)", "ROA (% p.a.)", "Fed funds rate (%)",
        ],
        "N": [
            df["ave_asset"].count(), df["total_asset"].count(),
            df["IncRate_w"].count(), df["ExpRate_w"].count(), df["ROA_w"].count(),
            df["ffrate"].count(),
        ],
        "Mean": [
            df["ave_asset"].mean() / 1e3, df["total_asset"].mean() / 1e3,
            df["IncRate_w"].mean(), df["ExpRate_w"].mean(), df["ROA_w"].mean(),
            df["ffrate"].mean(),
        ],
        "SD": [
            df["ave_asset"].std() / 1e3, df["total_asset"].std() / 1e3,
            df["IncRate_w"].std(), df["ExpRate_w"].std(), df["ROA_w"].std(),
            df["ffrate"].std(),
        ],
        "p1": [
            df["ave_asset"].quantile(0.01) / 1e3, df["total_asset"].quantile(0.01) / 1e3,
            df["IncRate_w"].quantile(0.01), df["ExpRate_w"].quantile(0.01),
            df["ROA_w"].quantile(0.01), df["ffrate"].quantile(0.01),
        ],
        "Median": [
            df["ave_asset"].median() / 1e3, df["total_asset"].median() / 1e3,
            df["IncRate_w"].median(), df["ExpRate_w"].median(), df["ROA_w"].median(),
            df["ffrate"].median(),
        ],
        "p99": [
            df["ave_asset"].quantile(0.99) / 1e3, df["total_asset"].quantile(0.99) / 1e3,
            df["IncRate_w"].quantile(0.99), df["ExpRate_w"].quantile(0.99),
```

```
            df["ROA_w"].quantile(0.99), df["ffrate"].quantile(0.99),
        ],
    }
)
tab = tab.round(3)
tab.to_csv(f"{OUT_TAB}/table1_descriptives.csv", index=False)
print(tab.to_string(index=False))

# ---------------------------------------------------------------- Fig 1
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
plt.rcParams.update({"font.size": 10, "figure.dpi": 150})

fig, axes = plt.subplots(1, 3, figsize=(13, 3.6))
g = df.groupby("qdate_dt").agg(
    n_banks=("idrssd", "nunique"),
    tot_asset=("total_asset", "sum"),
    med_asset=("total_asset", "median"),
)
axes[0].plot(g.index, g["n_banks"])
axes[0].set_title("Number of reporting banks")
axes[0].set_xlabel("Year")
g2 = df.groupby("qdate_dt")["total_asset"].sum() / 1e9  # $tn (raw unit: $ thousand)
axes[1].plot(g2.index, g2)
axes[1].set_title("Total assets ($tn)")
axes[1].set_xlabel("Year")
axes[2].plot(g.index, g["med_asset"] / 1e3)
axes[2].set_title("Median bank assets ($m)")
axes[2].set_xlabel("Year")
for a in axes:
    a.grid(alpha=0.3)
fig.tight_layout()
fig.savefig(f"{OUT_FIG}/fig1_industry_over_time.png", bbox_inches="tight")
plt.close(fig)

# ---------------------------------------------------------------- Fig 2
# missingness by year (share of bank-quarter rows with missing flows)
# computed on the raw panel BEFORE dropping rows
raw = pd.read_stata(
    f"{DATA}/callreports_1984_2017.dta",
    columns=["year", "interest_income", "interest_expense", "net_income"],
    convert_categoricals=False,
)
raw["any_missing"] = raw[["interest_income", "interest_expense", "net_income"]].isna().any(axis=1)
miss = raw.groupby("year")["any_missing"].mean() * 100
fig, ax = plt.subplots(figsize=(6.5, 3.4))
ax.bar(miss.index, miss.values)
ax.set_title("Share of bank-quarters with missing income/expense items (%)")
ax.set_xlabel("Year")
ax.grid(alpha=0.3, axis="y")
fig.tight_layout()
fig.savefig(f"{OUT_FIG}/fig2_missing_values.png", bbox_inches="tight")
plt.close(fig)

# ---------------------------------------------------------------- Fig 3
# One trend figure for the data-audit section: annual median IncRate, ExpRate, NIM, ROA.
med = df.groupby("year")[["IncRate_w", "ExpRate_w", "ROA_w"]].median()
df["NIM_w"] = df["IncRate_w"] - df["ExpRate_w"]
```

```
med["NIM"] = df.groupby("year")["NIM_w"].median()
fig, ax = plt.subplots(figsize=(8.5, 4.2))
ax.plot(med.index, med["IncRate_w"], lw=1.5, label="Median IncRate")
ax.plot(med.index, med["ExpRate_w"], lw=1.5, label="Median ExpRate")
ax.plot(med.index, med["NIM"], lw=1.5, label="Median NIM (IncRate - ExpRate)")
ax.plot(med.index, med["ROA_w"], lw=1.5, ls="--", label="Median ROA")
ax.set_title("Median bank rates over time")
ax.set_ylabel("% per annum")
ax.legend(fontsize=8)
ax.grid(alpha=0.3)
fig.tight_layout()
fig.savefig(f"{OUT_FIG}/fig3_rate_trends.png", bbox_inches="tight")
plt.close(fig)

# ---------------------------------------------------------------- save
df.to_parquet(f"{PROC}/bank_panel.parquet", index=False)
pd.DataFrame({
    "metric": ["raw_rows", "raw_banks", "clean_rows", "clean_banks", "dropped_rows"],
    "value": [before, raw_banks, len(df), df["idrssd"].nunique(), before - len(df)],
}).to_csv(PROC / "sample_summary.csv", index=False)
print("\nSaved bank_panel.parquet:", df.shape)
print("Banks:", df["idrssd"].nunique(), " Period:", df["year"].min(), "-", df["year"].max())
```

Listing 2: `02_estimate_betas.py` — Estimate bank-level interest-income, interest-expense, and ROA betas by a distributed-lag regression on federal-funds changes.

```
"""
Bank Interest-Rate Sensitivity
Script 02: Bank-level interest income / expense / ROA betas

For bank i and quarter t:
    d_y = y_it - y_i,t-1            (first difference within bank)
    d_f = f_t - f_t-1               (federal funds rate, quarter-end month)

    d_y_it = a_i + sum_{tau=0..3} b_i,tau * d_f_{t-tau} + u_it
    beta_i = b_i0 + b_i1 + b_i2 + b_i3

y in {IncRate_w, ExpRate_w, ROA_w} (winsorized annualized % rates).
Eligibility: bank must have >= N_MIN usable d_y observations
(calendar-quarter contiguous diffs with non-missing y_t and y_{t-1}).

Outputs:
  - data_processed/bank_betas.csv        (one row per eligible bank)
  - output/tables/beta_summary.csv       (summary of bank-level estimates)
  - output/figures/fig4_beta_distributions.png

Note on naming: the internal columns keep the _w suffix (they were estimated
from winsorized rates), but the output CSV and the report use beta_Inc,
beta_Exp, beta_ROA. The cleaned dataset has "one row for each bank and that bank's
estimated interest income and interest expense betas"; I keep all banks in the
file, with betas missing for banks that do not meet the observation minimum.
"""
import numpy as np
import pandas as pd
from pathlib import Path
from multiprocessing import Pool

```

```
ROOT = Path(__file__).resolve().parents[1]
PROC = ROOT / "data_processed"
OUT_TAB = ROOT / "output" / "tables"
OUT_FIG = ROOT / "output" / "figures"
for p in [PROC, OUT_TAB, OUT_FIG]:
    p.mkdir(parents=True, exist_ok=True)

N_MIN = 20   # minimum usable quarterly observations per bank (5 years)
YVARS = ["IncRate_w", "ExpRate_w", "ROA_w"]
# internal -> public names for the beta columns
BETA_NAMES = {"IncRate_w": "beta_Inc", "ExpRate_w": "beta_Exp", "ROA_w": "beta_ROA"}

# ------------------------------------------------------------------ load
df = pd.read_parquet(f"{PROC}/bank_panel.parquet")
df = df.sort_values(["idrssd", "qdate"]).reset_index(drop=True)

# --- quarterly fed funds changes (common across banks) ----------------
ff_by_q = df.drop_duplicates("qindex")[["qindex", "ffrate"]].set_index("qindex")["ffrate"]
calendar_q = np.arange(int(df["qindex"].min()), int(df["qindex"].max()) + 1)
ff_by_q = ff_by_q.reindex(calendar_q)
dff = ff_by_q.diff()
# attach lags 0..3 to every bank-quarter
for lag in range(4):
    df[f"dff_l{lag}"] = df["qindex"].map(dff.shift(lag))

# --- first differences of y within bank -------------------------------
# IMPORTANT: only keep a first difference if the two quarters are ADJACENT
# (qindex difference == 1). Banks sometimes skip a quarter; a naive shift(1)
# difference would then span two quarters of y while the fed funds change
# dff_l0 still covers one quarter, mismatching the two sides of the
# regression. I found this by spotting a few banks whose d_y jumped by more
# than any single-quarter rate change could explain.
dq = df.groupby("idrssd")["qindex"].diff()
adjacent = dq == 1
for v in YVARS:
    prev = df.groupby("idrssd")[v].shift(1)
    df[f"d_{v}"] = np.where(adjacent & prev.notna(), df[v] - prev, np.nan)

# --- per-bank regression ----------------------------------------------
bank_ids = df["idrssd"].unique()

def fit_bank(bid):
    b = df[df["idrssd"] == bid]
    res = {"idrssd": bid}
    for v in YVARS:
        y = b[f"d_{v}"].to_numpy(float)
        X = np.column_stack([
            np.ones(len(b)),
            b["dff_l0"].to_numpy(float),
            b["dff_l1"].to_numpy(float),
            b["dff_l2"].to_numpy(float),
            b["dff_l3"].to_numpy(float),
        ])
        ok = ~(np.isnan(y) | np.isnan(X).any(axis=1))
        y, X = y[ok], X[ok]
        n = len(y)
        res[f"n_{v}"] = n
        if n < N_MIN or np.linalg.matrix_rank(X) < 5:
            res[f"beta_{v}"] = np.nan
```

```
            res[f"r2_{v}"] = np.nan
            continue
        coef, *_ = np.linalg.lstsq(X, y, rcond=None)
        beta = coef[1:5].sum()
        # R2
        yhat = X @ coef
        ss_res = np.sum((y - yhat) ** 2)
        ss_tot = np.sum((y - y.mean()) ** 2)
        res[f"beta_{v}"] = beta
        res[f"r2_{v}"] = 1 - ss_res / ss_tot if ss_tot > 0 else np.nan
    return res

if __name__ == "__main__":
    # bank attributes for reporting
    attrs = (
        df.groupby("idrssd")
        .agg(
            n_quarters=("qdate", "count"),
            first_q=("qdate", "min"),
            last_q=("qdate", "max"),
            mean_asset=("total_asset", "mean"),
            median_asset=("total_asset", "median"),
            max_asset=("total_asset", "max"),
            bank_name=("bank_name", "last"),
            bhc_id=("bhc_id", "last"),
        )
        .reset_index()
    )

    with Pool(8) as pool:
        rows = pool.map(fit_bank, bank_ids, chunksize=200)

    betas = pd.DataFrame(rows).merge(attrs, on="idrssd", how="left")

    # rename beta columns to the public names (drop _w in output/report)
    betas = betas.rename(columns={
        f"beta_{v}": BETA_NAMES[v] for v in YVARS
    })
    # keep n_* / r2_* under public names as well for readability
    betas = betas.rename(columns={
        f"n_{v}": f"n_{BETA_NAMES[v]}" for v in YVARS
    })

    # eligibility for each outcome: at least one beta non-missing
    for v in YVARS:
        betas[f"eligible_{BETA_NAMES[v]}"] = betas[BETA_NAMES[v]].notna()
    betas["eligible_any"] = betas[[f"eligible_{BETA_NAMES[v]}" for v in YVARS]].any(axis=1)

    betas.to_csv(f"{PROC}/bank_betas.csv", index=False)
    print("banks with any beta:", betas["eligible_any"].sum(), "/", len(betas))
    for v in YVARS:
        print(f"  eligible_{BETA_NAMES[v]}: {betas[f'eligible_{BETA_NAMES[v]}'].sum()}")

    # ---------------------------------------------------------- summary
    summ = pd.DataFrame(
        {
            "Statistic": ["N banks", "Mean", "SD", "p10", "p25", "Median", "p75", "p90"],
            "Beta_Inc": [
                int(betas["beta_Inc"].notna().sum()),
```

```
                betas["beta_Inc"].mean(),
                betas["beta_Inc"].std(),
                betas["beta_Inc"].quantile(0.10),
                betas["beta_Inc"].quantile(0.25),
                betas["beta_Inc"].median(),
                betas["beta_Inc"].quantile(0.75),
                betas["beta_Inc"].quantile(0.90),
            ],
            "Beta_Exp": [
                int(betas["beta_Exp"].notna().sum()),
                betas["beta_Exp"].mean(),
                betas["beta_Exp"].std(),
                betas["beta_Exp"].quantile(0.10),
                betas["beta_Exp"].quantile(0.25),
                betas["beta_Exp"].median(),
                betas["beta_Exp"].quantile(0.75),
                betas["beta_Exp"].quantile(0.90),
            ],
            "Beta_ROA": [
                int(betas["beta_ROA"].notna().sum()),
                betas["beta_ROA"].mean(),
                betas["beta_ROA"].std(),
                betas["beta_ROA"].quantile(0.10),
                betas["beta_ROA"].quantile(0.25),
                betas["beta_ROA"].median(),
                betas["beta_ROA"].quantile(0.75),
                betas["beta_ROA"].quantile(0.90),
            ],
        }
    )
    summ = summ.round(3)
    # Keep the count row as an integer when serialized for the report table.
    for col in ["Beta_Inc", "Beta_Exp", "Beta_ROA"]:
        summ[col] = summ[col].astype(object)
        summ.at[0, col] = str(int(summ.at[0, col]))
    summ.to_csv(f"{OUT_TAB}/beta_summary.csv", index=False)
    print("\n", summ.to_string(index=False))

    # ---------------------------------------------------------- figure
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    plt.rcParams.update({"font.size": 10, "figure.dpi": 150})

    fig, axes = plt.subplots(1, 3, figsize=(12, 3.6))
    for ax, v, ttl in zip(
        axes,
        ["beta_Inc", "beta_Exp", "beta_ROA"],
        ["Interest income beta", "Interest expense beta", "ROA beta"],
    ):
        s = betas[v].dropna()
        ax.hist(s, bins=60, color="steelblue", alpha=0.8)
        ax.axvline(s.median(), color="crimson", ls="--", lw=1, label=f"median {s.median():.2f}")
        ax.set_title(ttl)
        ax.set_xlabel("Beta")
        ax.legend(fontsize=8)
        ax.grid(alpha=0.3)
    fig.tight_layout()
    fig.savefig(f"{OUT_FIG}/fig4_beta_distributions.png", bbox_inches="tight")
```

```
    plt.close(fig)
    print("done.")
```

Listing 3: `02b_asym_betas.py` — Split federal-funds changes into positive and negative parts and estimate asymmetric expense betas for the full and pre-2007 samples.

```
"""
Bank Interest-Rate Sensitivity
Script 02b: Asymmetric expense-rate betas

This script tests whether the pass-through of policy-rate changes to bank
funding costs is symmetric in rate increases versus rate decreases. I split
the quarterly federal-funds change into an increase part and a decrease part,

    dff_pos = max(dff, 0),   dff_neg = min(dff, 0),

and for each bank estimate

    d_ExpRate_it = a_i + sum_{tau=0..3} b_up_itau * dff_pos_{t-tau}
                       + sum_{tau=0..3} b_down_itau * dff_neg_{t-tau} + u_it

    beta_Exp_up   = sum_tau b_up_itau
    beta_Exp_down = sum_tau b_down_itau

Both sums capture the cumulative response of the funding-cost rate to four
quarterly changes in the policy rate, exactly as in Script 02, but separately
for rate hikes and cuts.

Only adjacent-quarter differences of ExpRate are used (same rule as script
02). A bank needs at least N_MIN usable observations to keep its estimates;
I use 28 (7 years) rather than 20 because splitting the regressors roughly
halves the effective variation on each side. Estimated betas are not trimmed.

The script runs the same estimation twice:
  - the full sample (1984Q1-2017Q4), saved as bank_betas_asym.csv;
  - a pre-2007 sample that truncates the panel at 2007Q4 (qdate SAMPLE_END)
    before constructing any difference, saved as bank_betas_asym_pre07.csv.
The pre-2007 version exists because the asymmetric beta estimated on the
full sample would embed post-2007 information (including the zero-lower-bound
period) and could not separately identify whether the asymmetry predates
2008. Truncating the data before estimating avoids that look-ahead: the
pre-2007 betas use only rate changes observed through 2007Q4.

Outputs:
  - data_processed/bank_betas_asym.csv        (full sample)
  - data_processed/bank_betas_asym_pre07.csv  (qdate <= 2007Q4 only)
  both with columns (idrssd, beta_Exp_up, beta_Exp_down, n_obs)
"""
import numpy as np
import pandas as pd
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
PROC = ROOT / "data_processed"
PROC.mkdir(parents=True, exist_ok=True)

N_MIN = 28       # minimum usable observations for the asymmetric regression
```

```python
SAMPLE_END = 2007 * 4 + 3  # calendar-quarter index for 2007Q4

df = pd.read_parquet(f"{PROC}/bank_panel.parquet")
df = df.sort_values(["idrssd", "qdate"]).reset_index(drop=True)


def estimate_sample(sample, out_path, tag):
    """Run the asymmetric regression on one truncated panel and save it."""
    sample = sample.copy()

    # --- quarterly fed funds changes within this sample, split pos / neg --
    ff_by_q = sample.drop_duplicates("qindex")[["qindex", "ffrate"]].set_index("qindex")["ffrate"]
    calendar_q = np.arange(int(sample["qindex"].min()), int(sample["qindex"].max()) + 1)
    ff_by_q = ff_by_q.reindex(calendar_q)
    dff = ff_by_q.diff()
    dff_pos = dff.clip(lower=0.0)
    dff_neg = dff.clip(upper=0.0)
    for lag in range(4):
        sample[f"dff_pos_l{lag}"] = sample["qindex"].map(dff_pos.shift(lag))
        sample[f"dff_neg_l{lag}"] = sample["qindex"].map(dff_neg.shift(lag))

    # --- first differences of ExpRate within bank (adjacent quarters only)
    dq = sample.groupby("idrssd")["qindex"].diff()
    adjacent = dq == 1
    prev = sample.groupby("idrssd")["ExpRate_w"].shift(1)
    sample["d_ExpRate_w"] = np.where(
        adjacent & prev.notna(), sample["ExpRate_w"] - prev, np.nan
    )

    rows = []
    for bid, b in sample.groupby("idrssd", sort=False):
        y = b["d_ExpRate_w"].to_numpy(float)
        X = np.column_stack([
            np.ones(len(b)),
            b["dff_pos_l0"].to_numpy(float), b["dff_pos_l1"].to_numpy(float),
            b["dff_pos_l2"].to_numpy(float), b["dff_pos_l3"].to_numpy(float),
            b["dff_neg_l0"].to_numpy(float), b["dff_neg_l1"].to_numpy(float),
            b["dff_neg_l2"].to_numpy(float), b["dff_neg_l3"].to_numpy(float),
        ])
        ok = ~(np.isnan(y) | np.isnan(X).any(axis=1))
        y, X = y[ok], X[ok]
        n = len(y)
        out = {"idrssd": bid, "beta_Exp_up": np.nan, "beta_Exp_down": np.nan, "n_obs": n}
        if n < N_MIN or np.linalg.matrix_rank(X) < 9:
            rows.append(out)
            continue
        coef, *_ = np.linalg.lstsq(X, y, rcond=None)
        out["beta_Exp_up"] = coef[1:5].sum()
        out["beta_Exp_down"] = coef[5:9].sum()
        rows.append(out)

    asym = pd.DataFrame(rows)
    asym.to_csv(out_path, index=False)
    ok = asym.dropna(subset=["beta_Exp_up", "beta_Exp_down"])
    print(f"[{tag}] banks: {len(asym)}, with both betas: {len(ok)}, "
          f"median up {ok['beta_Exp_up'].median():.3f}, "
          f"median down {ok['beta_Exp_down'].median():.3f}")
    print("saved", out_path)

```

```
111
112 if __name__ == "__main__":
113     estimate_sample(df, f"{PROC}/bank_betas_asym.csv", "full sample")
114     pre07 = df[df["qindex"] <= SAMPLE_END].copy()
115     estimate_sample(pre07, f"{PROC}/bank_betas_asym_pre07.csv", "pre-2007")
```

Listing 4: `03_cross_section.py` — Cross-sectional regressions of income and ROA betas on expense beta, with mean-assets terciles and a demeaned log-asset interaction.

```
"""
Bank Interest-Rate Sensitivity
Script 03: Cross-sectional regressions

A: beta_Inc = alpha + gamma * beta_Exp + e
B: beta_ROA = alpha + gamma * beta_Exp + e

- OLS with heteroskedasticity-robust (HC1) standard errors.
- Size heterogeneity: split banks into terciles by average total assets;
  also an interacted version with demeaned log mean assets.
- Figures: binned scatter plots (20 bins of beta_Exp) with OLS line.

Outputs:
  - output/tables/table2_cross_section.csv
  - output/figures/fig5_inc_vs_exp.png
  - output/figures/fig6_roa_vs_exp.png

Size terciles are defined by each bank's time-series average total assets
(mean_asset), not within-quarter assets.
"""
import numpy as np
import pandas as pd
import statsmodels.api as sm
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
PROC = ROOT / "data_processed"
OUT_TAB = ROOT / "output" / "tables"
OUT_FIG = ROOT / "output" / "figures"
for p in [PROC, OUT_TAB, OUT_FIG]:
    p.mkdir(parents=True, exist_ok=True)

betas = pd.read_csv(f"{PROC}/bank_betas.csv")

# analysis sample: banks with both income and expense betas
A = betas.dropna(subset=["beta_Inc", "beta_Exp"]).copy()
B = betas.dropna(subset=["beta_ROA", "beta_Exp"]).copy()

# log mean assets and size terciles (whole distribution of mean assets)
# NOTE: mean_asset is each bank's TIME-SERIES AVERAGE of total assets over its
# usable sample (computed in script 02), not assets at one point in time. With
# one row per bank, this is the natural size measure; it smooths out quarterly
# swings but also means size is not pinned to a single reference quarter.
for d in (A, B):
    d["log_asset"] = np.log(d["mean_asset"])
    d["log_asset_c"] = d["log_asset"] - d["log_asset"].mean()   # demeaned
    d["size_tercile"] = pd.qcut(d["mean_asset"], 3, labels=["Small", "Medium", "Large"])

```

```python
# ------------------------------------------------------------------ A
def ols_table(y, x, d, label):
    X = sm.add_constant(d[x])
    m = sm.OLS(d[y], X).fit(cov_type="HC1")
    n = int(m.nobs)
    ar2 = float(m.rsquared_adj)
    return {
        "Specification": label,
        "N": n,
        "Intercept": m.params["const"],
        "Intercept_t": m.tvalues["const"],
        "Slope": m.params[x],
        "Slope_t": m.tvalues[x],
        "Adj_R2": ar2,
    }

rows = [
    ols_table("beta_Inc", "beta_Exp", A, "A: Inc on Exp (full)"),
    ols_table("beta_ROA", "beta_Exp", B, "B: ROA on Exp (full)"),
]
# size terciles - Section A
for t in ["Small", "Medium", "Large"]:
    d = A[A["size_tercile"] == t]
    rows.append(ols_table("beta_Inc", "beta_Exp", d, f"A: Inc on Exp, {t}"))
for t in ["Small", "Medium", "Large"]:
    d = B[B["size_tercile"] == t]
    rows.append(ols_table("beta_ROA", "beta_Exp", d, f"B: ROA on Exp, {t}"))

# interaction with size (demeaned log assets)
# The expense-beta main effect is evaluated at the sample-average bank size
# because log_asset is demeaned before interacting.
for d, y, tag in [(A, "beta_Inc", "A"), (B, "beta_ROA", "B")]:
    X = sm.add_constant(
        pd.DataFrame(
            {
                "Exp": d["beta_Exp"],
                "Exp_x_logA": d["beta_Exp"] * d["log_asset_c"],
                "log_asset_c": d["log_asset_c"],
            }
        )
    )
    m = sm.OLS(d[y], X).fit(cov_type="HC1")
    rows.append(
        {
            "Specification": f"{tag}: interaction with log assets",
            "N": int(m.nobs),
            "Intercept": m.params["const"],
            "Intercept_t": m.tvalues["const"],
            "Slope": m.params["Exp"],
            "Slope_t": m.tvalues["Exp"],
            "Int_coef": m.params["Exp_x_logA"],
            "Int_t": m.tvalues["Exp_x_logA"],
            "Adj_R2": float(m.rsquared_adj),
        }
    )
# ensure the plain rows have the extra columns too
for r in rows:
    r.setdefault("Int_coef", np.nan)
    r.setdefault("Int_t", np.nan)
```

```

t2 = pd.DataFrame(rows).round(3)
t2.to_csv(f"{OUT_TAB}/table2_cross_section.csv", index=False)
print(t2.to_string(index=False))

# ------------------------------------------------------------ figures
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
plt.rcParams.update({"font.size": 10, "figure.dpi": 150})

def binned_scatter(d, y, x, ax, title, nbins=20):
    d = d.dropna(subset=[y, x])
    # 20 equal-frequency bins of x
    d = d.assign(bin=pd.qcut(d[x], nbins, duplicates="drop"))
    g = d.groupby("bin", observed=True).agg(xm=(x, "mean"), ym=(y, "mean"), n=(y, "size"))
    ax.scatter(g["xm"], g["ym"], s=28, color="steelblue", label="Binned means")
    X = sm.add_constant(d[x])
    m = sm.OLS(d[y], X).fit(cov_type="HC1")
    xs = np.linspace(d[x].min(), d[x].max(), 100)
    ax.plot(xs, m.params["const"] + m.params[x] * xs, color="crimson", lw=1.5,
            label=f"OLS slope {m.params[x]:.2f} (HC1 t={m.tvalues[x]:.1f})")
    ax.set_title(title)
    ax.set_xlabel(x)
    ax.set_ylabel(y)
    ax.legend(fontsize=8)
    ax.grid(alpha=0.3)

fig, ax = plt.subplots(figsize=(6.2, 4.2))
binned_scatter(A, "beta_Inc", "beta_Exp", ax,
               "Income beta vs. expense beta (binned)")
fig.tight_layout()
fig.savefig(f"{OUT_FIG}/fig5_inc_vs_exp.png", bbox_inches="tight")
plt.close(fig)

fig, ax = plt.subplots(figsize=(6.2, 4.2))
binned_scatter(B, "beta_ROA", "beta_Exp", ax,
               "ROA beta vs. expense beta (binned)")
fig.tight_layout()
fig.savefig(f"{OUT_FIG}/fig6_roa_vs_exp.png", bbox_inches="tight")
plt.close(fig)
print("figures saved.")
```

Listing 5: `04_stock_link.py` — Link banks to CRSP PERMCOs, aggregate share classes by prior-month market cap, build the total-return index, and run stock-return regressions.

```
"""
Bank Interest-Rate Sensitivity
Script 04: Linking banks to CRSP and stock-price analysis

Steps:
 1. Map call-report banks to CRSP PERMCO via Bank_and_CRSP_Linking_Table.csv,
    matching on idrssd first and bhc_id second, restricted to the link
    validity window [dt_start, dt_end]. Report the number of matched banks
    and distinct listed companies, and the number of bank-quarters that map
    to more than one PERMCO (I checked these; see README).
 2. Aggregate multiple PERMNO share classes of the same company-month into
```

```
    one company-month record using market-capitalization weights.
 3. Build an equal-weighted and a value-weighted TOTAL RETURN index for
    publicly traded banks (2000-2017, the overlap of CRSP and call reports).
    The index uses MthRet, which includes dividends.
 4. Aggregate bank accounting data to the listed-company (PERMCO) level
    (sum assets, income and expense flows; recompute and quarter-winsorize rates) and regress
    monthly stock returns on LAGGED company accounting levels (not on
    company-level betas, which would embed post-return information).

Accounting-to-return timing: a quarter-q report is treated as public from
two months after the quarter end (Call Report filing lags). Returns earned
before that date are matched to the previous quarter's data, ensuring
accounting data used in a regression were already public.

The company-level expense beta is not used in return regressions because a
full-sample beta would embed post-return information; return tests use only
lagged accounting levels.

Outputs:
  - data_processed/crsp_link_summary.csv
  - data_processed/company_panel.parquet
  - output/tables/table4_stock_regressions.csv
  - output/figures/fig7_bank_stock_index.png
"""
import numpy as np
import pandas as pd
import statsmodels.api as sm
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
DATA = ROOT / "data"
PROC = ROOT / "data_processed"
OUT_TAB = ROOT / "output" / "tables"
OUT_FIG = ROOT / "output" / "figures"
for p in [PROC, OUT_TAB, OUT_FIG]:
    p.mkdir(parents=True, exist_ok=True)

# ---------------------------------------------------------------- load
bank = pd.read_parquet(f"{PROC}/bank_panel.parquet")
lk = pd.read_csv(f"{DATA}/Bank_and_CRSP_Linking_Table.csv")

# Read the raw CRSP CSV directly (not a cached parquet), keeping only the
# columns needed for the analysis. ~2.4M rows; the read takes a minute or two.
CRSP_COLS = ["PERMNO", "PERMCO", "YYYYMM", "MthCalDt", "MthPrc", "MthRet",
             "MthCap", "MthVol", "vwretd", "IssuerNm"]
cr = pd.read_csv(f"{DATA}/CRSP_Monthly_Stock_File.csv", usecols=CRSP_COLS)

# PERMCO as a robust integer (some rows can be blank/non-numeric in CRSP)
cr["permco"] = pd.to_numeric(cr["PERMCO"], errors="coerce")
cr = cr.dropna(subset=["permco"]).copy()
cr["permco"] = cr["permco"].astype("int64")
cr["month_dt"] = pd.to_datetime(cr["YYYYMM"].astype(str) + "01")

# ---------------------------------------------------------------- link
# Priority rule: match on the bank's own idrssd first; only banks that do NOT
# match on idrssd are then tried via their holding company's bhc_id. This
# avoids a bank being mapped to two different listed companies (once through
# its own idrssd, once through its BHC), which would double-count its assets
# and flows when aggregating to the company level.
```

```
lk["dt_start"] = pd.to_datetime(lk["dt_start"])
lk["dt_end"] = pd.to_datetime(lk["dt_end"])

bank_key = bank[["idrssd", "bhc_id", "qdate", "qdate_dt"]].drop_duplicates()

m1 = bank_key.merge(
    lk.rename(columns={"rssd9001": "idrssd"}), on="idrssd", how="inner")
m1 = m1[(m1["qdate_dt"] >= m1["dt_start"]) & (m1["qdate_dt"] <= m1["dt_end"])]
m1["match_type"] = "idrssd"

# Try the holding-company identifier for bank-quarters without a valid
# own-id link. The fallback is quarter-specific: an own-id match in one period
# must not block a valid BHC match in another period.
own_keys = m1[["idrssd", "qdate"]].drop_duplicates()
bank_bhc = bank_key.merge(own_keys, on=["idrssd", "qdate"], how="left", indicator=True)
bank_bhc = bank_bhc.loc[bank_bhc["_merge"].eq("left_only")].drop(columns="_merge")
bank_bhc = bank_bhc.dropna(subset=["bhc_id"]).rename(columns={"bhc_id": "rssd9001"})
m2 = bank_bhc.merge(lk, on="rssd9001", how="inner").rename(columns={"rssd9001": "bhc_id"})
m2 = m2[(m2["qdate_dt"] >= m2["dt_start"]) & (m2["qdate_dt"] <= m2["dt_end"])]
m2["match_type"] = "bhc_id"

merged = pd.concat([m1[["idrssd", "qdate", "permco", "match_type"]],
                    m2[["idrssd", "qdate", "permco", "match_type"]]], ignore_index=True)

# Prefer own-id links over BHC links. If multiple PERMCOs remain at the same
# priority for one bank-quarter, the link is ambiguous, so exclude that link
# instead of assigning the bank to an arbitrary company.
conf = merged.groupby(["idrssd", "qdate"])["permco"].nunique()
n_conflict_q = int((conf > 1).sum())
print("bank-quarters with >1 PERMCO before resolution:", n_conflict_q)
merged["priority"] = merged["match_type"].map({"idrssd": 0, "bhc_id": 1})
best = merged.groupby(["idrssd", "qdate"])["priority"].transform("min")
best_matches = merged.loc[merged["priority"].eq(best)].copy()
ambiguous = best_matches.groupby(["idrssd", "qdate"])["permco"].nunique()
ambiguous_keys = ambiguous[ambiguous > 1].reset_index()[["idrssd", "qdate"]]
best_matches = best_matches.merge(ambiguous_keys.assign(_ambiguous=True),
                                  on=["idrssd", "qdate"], how="left")
best_matches = best_matches.loc[best_matches["_ambiguous"].isna()]
merged = best_matches.drop_duplicates(["idrssd", "qdate"])[
    ["idrssd", "qdate", "permco", "match_type"]
].copy()
n_ambiguous_q = len(ambiguous_keys)
print("ambiguous bank-quarters excluded:", n_ambiguous_q)

n_banks_matched = merged["idrssd"].nunique()
n_permco = merged["permco"].nunique()
print(f"matched bank-quarters: {len(merged)}, banks: {n_banks_matched}, permco: {n_permco}")
pd.DataFrame({"metric": ["banks_matched", "permco_distinct", "bank_quarter_links",
                         "conflict_bank_quarters", "ambiguous_bank_quarters_excluded"],
              "value": [n_banks_matched, n_permco, len(merged), n_conflict_q, n_ambiguous_q]}
             ).to_csv(f"{PROC}/crsp_link_summary.csv", index=False)

# ---------------------------------------------------------- stock returns
# Collapse multiple PERMNO share classes into a company return using the
# beginning-of-month market capitalization weights. Contemporaneous weights
# would use information realized during the return month.
dup = cr.duplicated(["permco", "month_dt"]).sum()
print("permco-month duplicates:", dup)
cr["MthCap"] = pd.to_numeric(cr["MthCap"], errors="coerce")
```

```
cr["month_period"] = cr["month_dt"].dt.to_period("M")
company_cap = cr.groupby(["permco", "month_dt"], as_index=False)["MthCap"].sum()
cr = cr.sort_values(["PERMNO", "month_dt"])
cr["cap_lag"] = cr.groupby("PERMNO")["MthCap"].shift(1)
cr["prior_month"] = cr.groupby("PERMNO")["month_period"].shift(1)
cr = cr.loc[cr["prior_month"] + 1 == cr["month_period"]].copy()
cr_d = cr.dropna(subset=["MthCap", "MthRet", "cap_lag"])
cr_d = cr_d.loc[cr_d["cap_lag"] > 0].copy()
cr_d["weighted_ret"] = cr_d["MthRet"] * cr_d["cap_lag"]
comp_month = cr_d.groupby(["permco", "month_dt"]).agg(
    weighted_ret=("weighted_ret", "sum"),
    cap_lag=("cap_lag", "sum"),
    vwretd=("vwretd", "first"),
    IssuerNm=("IssuerNm", "first"),
).reset_index()
comp_month["MthRet"] = comp_month["weighted_ret"] / comp_month["cap_lag"]
comp_month = comp_month.merge(company_cap, on=["permco", "month_dt"], how="left")
comp_month["month_period"] = comp_month["month_dt"].dt.to_period("M")
comp_month = comp_month.sort_values(["permco", "month_dt"])
comp_month["prior_month"] = comp_month.groupby("permco")["month_period"].shift(1)
comp_month["cap_lag_company"] = comp_month.groupby("permco")["MthCap"].shift(1)
comp_month.loc[comp_month["prior_month"] + 1 != comp_month["month_period"],
               "cap_lag_company"] = np.nan

stk = comp_month.copy()
stk = stk.dropna(subset=["MthRet"])
stk = stk[(stk["month_dt"] >= "2000-01-01") & (stk["month_dt"] <= "2017-12-31")]

# ------------------------------------------------------------ price index
# MthRet is the total (dividend-inclusive) monthly return, so the compounded
# index is a total-return index.
bank_permcos = merged["permco"].unique()
bstk = stk[stk["permco"].isin(bank_permcos)].copy().sort_values(["permco", "month_dt"])
ew = bstk.groupby("month_dt")["MthRet"].mean().sort_index()
bstk2 = bstk.sort_values(["permco", "month_dt"])
bstk2["cap_lag"] = bstk2["cap_lag_company"]
def wsum(d):
    w = d["cap_lag"] / d["cap_lag"].sum()
    return (w * d["MthRet"]).sum()
vw = (bstk2.dropna(subset=["cap_lag"]).groupby("month_dt").apply(wsum, include_groups=False).
    sort_index())

idx_ew = (1 + ew).cumprod() * 100
idx_vw = (1 + vw).cumprod() * 100

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
plt.rcParams.update({"font.size": 10, "figure.dpi": 150})
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(idx_ew.index, idx_ew.values, label="Equal-weighted", lw=1.3)
ax.plot(idx_vw.index, idx_vw.values, label="Value-weighted", lw=1.3)
ax.set_title("Total return index of publicly traded banks (2000 = 100, dividends included)")
ax.set_ylabel("Index")
ax.legend()
ax.grid(alpha=0.3)
fig.tight_layout()
fig.savefig(f"{OUT_FIG}/fig7_bank_stock_index.png", bbox_inches="tight")
plt.close(fig)
```

```
print(f"index: n months {len(idx_ew)}, last EW {idx_ew.iloc[-1]:.0f}, VW {idx_vw.iloc[-1]:.0f}")

# ------------------------------------------------ company aggregation
acc = bank[["idrssd", "qdate", "qdate_dt", "ave_asset", "total_asset",
            "interest_income_q", "interest_expense_q", "net_income_q"]]
acc_m = acc.merge(merged[["idrssd", "qdate", "permco"]], on=["idrssd", "qdate"], how="inner")
comp = (acc_m.groupby(["permco", "qdate", "qdate_dt"]).agg(
    ave_asset=("ave_asset", "sum"), total_asset=("total_asset", "sum"),
    interest_income_q=("interest_income_q", "sum"),
    interest_expense_q=("interest_expense_q", "sum"),
    net_income_q=("net_income_q", "sum"), n_banks=("idrssd", "nunique"),
).reset_index())
comp["IncRate"] = 400.0 * comp["interest_income_q"] / comp["ave_asset"]
comp["ExpRate"] = 400.0 * comp["interest_expense_q"] / comp["ave_asset"]
comp["ROA"] = 400.0 * comp["net_income_q"] / comp["ave_asset"]
# Match the bank-level treatment after recomputing consolidated company rates.
# The cutoffs are cross-sectional within quarter and affect Table 4 regressors.
for v in ["IncRate", "ExpRate", "ROA"]:
    comp[v] = comp.groupby("qdate")[v].transform(
        lambda s: s.clip(s.quantile(0.005), s.quantile(0.995))
    )
comp["log_asset"] = np.log(comp["total_asset"])
comp.to_parquet(f"{PROC}/company_panel.parquet")

# ---------------------------------------------------------- public timing
# quarter-end -> first month when report is assumed public (quarter end + 2 months)
comp["public_from"] = comp["qdate_dt"] + pd.offsets.MonthEnd(2) + pd.offsets.MonthBegin(1)
comp["public_ym"] = comp["public_from"].dt.year * 100 + comp["public_from"].dt.month
stk2 = stk.rename(columns={"month_dt": "month"}).copy()
stk2["permco"] = stk2["permco"].astype("int64")
stk2["month_ym"] = stk2["month"].dt.year * 100 + stk2["month"].dt.month

# keep only bank stocks for the asof match
bstk_panel = stk2[stk2["permco"].isin(merged["permco"].unique())].copy()
comp2 = comp[["permco", "public_ym", "log_asset", "IncRate", "ExpRate", "ROA",
              "n_banks"]].copy()

# manual asof: for each permco, latest public_from <= month
def asof_match(stk_g, comp_g):
    c = comp_g.sort_values("public_ym")
    cp = c["public_ym"].to_numpy()
    idx = np.searchsorted(cp, stk_g["month_ym"].to_numpy(), side="right") - 1
    ok = idx >= 0
    out = pd.DataFrame(index=stk_g.index)
    for col in ["log_asset", "IncRate", "ExpRate", "ROA", "n_banks"]:
        out[col] = np.nan
        out.loc[ok, col] = c[col].to_numpy()[idx[ok]]
    return out

parts = []
for permco, g in bstk_panel.groupby("permco", sort=False):
    cg = comp2[comp2["permco"] == permco]
    if len(cg) == 0:
        continue
    m = asof_match(g, cg)
    g2 = g.copy()
    for col in ["log_asset", "IncRate", "ExpRate", "ROA", "n_banks"]:
        g2[col] = m[col].to_numpy()
    parts.append(g2)
```

```

panel = pd.concat(parts, ignore_index=True)
panel = panel.dropna(subset=["log_asset", "MthRet"])
print("panel rows:", len(panel))

# ------------------------------------------------------------ regressions
# Wide table: one row per variable, one column per specification; each cell
# shows coef (t). Only lagged accounting LEVELS enter (not company betas).
VARS = ["const", "vwretd", "log_asset", "ROA", "IncRate", "ExpRate"]
VLAB = {"const": "Constant", "vwretd": "Market return (vwretd)",
        "log_asset": "Log total assets", "ROA": "ROA (%)",
        "IncRate": "Income rate (%)", "ExpRate": "Expense rate (%)"}

def run(y, X, d, label):
    m = sm.OLS(d[y], sm.add_constant(X)).fit(cov_type="cluster", cov_kwds={"groups": d["permco"]})
    return {"Spec": label, "N": int(m.nobs), "params": m.params, "tvalues": m.tvalues,
            "R2": float(m.rsquared)}

specs = []
d = panel.dropna(subset=["vwretd"])
specs.append(run("MthRet", d[["vwretd"]], d, "(1) Market"))
X = d[["vwretd", "log_asset", "ROA"]].dropna()
dd = d.loc[X.index]
specs.append(run("MthRet", dd[["vwretd", "log_asset", "ROA"]], dd, "(2) +Size+ROA"))
X3 = dd[["vwretd", "log_asset", "ROA", "IncRate", "ExpRate"]].dropna()
d3 = dd.loc[X3.index]
specs.append(run("MthRet", d3[["vwretd", "log_asset", "ROA", "IncRate", "ExpRate"]], d3, "(3) +
    Rates"))

for s in specs:
    print(s)

wide = pd.DataFrame({"Variable": [VLAB[v] for v in VARS]})
for s in specs:
    col = []
    for v in VARS:
        if v not in s["params"]:
            col.append("")
        else:
            col.append(f"{s['params'][v]:.4f} ({s['tvalues'][v]:.2f})")
    wide[s["Spec"]] = col
wide.loc[len(wide)] = ["N"] + [str(s["N"]) for s in specs]
wide.loc[len(wide)] = ["R-squared"] + [f"{s['R2']:.4f}" for s in specs]
wide.to_csv(f"{OUT_TAB}/table4_stock_regressions.csv", index=False)
print(wide.to_string(index=False))
print("done.")
```

Listing 6: `05_q2_extension.py` — Summarize and plot the hike-versus-cut asymmetry, with paired tests and size terciles for the full and pre-2007 samples.

```
"""Compare banks' expense-rate pass-through during hikes and cuts."""
from pathlib import Path

import pandas as pd
from scipy import stats
import matplotlib
matplotlib.use("Agg")
```

```python
import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parents[1]
PROC = ROOT / "data_processed"
OUT_TAB = ROOT / "output" / "tables"
OUT_FIG = ROOT / "output" / "figures"
for path in (PROC, OUT_TAB, OUT_FIG):
    path.mkdir(parents=True, exist_ok=True)

rows = []
def summarize(label, estimates):
    d = estimates.dropna(subset=["beta_Exp_up", "beta_Exp_down"]).copy()
    diff = d["beta_Exp_up"] - d["beta_Exp_down"]
    paired = stats.ttest_rel(d["beta_Exp_up"], d["beta_Exp_down"])
    return {
        "Sample": label,
        "N banks": int(len(d)),
        "Mean beta_Exp_up": d["beta_Exp_up"].mean(),
        "Mean beta_Exp_down": d["beta_Exp_down"].mean(),
        "Mean difference (up - down)": diff.mean(),
        "Median difference (up - down)": diff.median(),
        "Paired t-stat": paired.statistic,
        "Paired p-value": paired.pvalue,
        "Share with diff > 0": (diff > 0).mean(),
    }

full = pd.read_csv(PROC / "bank_betas_asym.csv")
pre07 = pd.read_csv(PROC / "bank_betas_asym_pre07.csv")
rows.append(summarize("Full sample (1984-2017)", full))

# Divide the full-sample asymmetric estimation sample into bank-size terciles
# using the time-series mean of total assets, the same measure as in Script 03.
size = pd.read_csv(PROC / "bank_betas.csv")[["idrssd", "mean_asset"]]
full_sized = full.merge(size, on="idrssd", how="left")
full_sized = full_sized.dropna(subset=["beta_Exp_up", "beta_Exp_down", "mean_asset"])
full_sized["Size tercile"] = pd.qcut(
    full_sized["mean_asset"], 3, labels=["Small", "Medium", "Large"]
)
for group in ["Small", "Medium", "Large"]:
    rows.append(summarize(group, full_sized[full_sized["Size tercile"] == group]))
rows.append(summarize("Pre-2007 (re-estimated)", pre07))

summary = pd.DataFrame(rows).round(4)
summary.to_csv(OUT_TAB / "table5_asymmetry.csv", index=False)
print(summary.to_string(index=False))

# Figure 8 compares the two cumulative betas directly; the 45-degree line
# represents equal pass-through. Plot limits use the central 99% of values so
# a handful of noisy estimates do not make the main pattern unreadable.
d = full.dropna(subset=["beta_Exp_up", "beta_Exp_down"])
low = min(d["beta_Exp_up"].quantile(0.005), d["beta_Exp_down"].quantile(0.005), 0)
high = max(d["beta_Exp_up"].quantile(0.995), d["beta_Exp_down"].quantile(0.995))
fig, ax = plt.subplots(figsize=(6.2, 5.2))
ax.scatter(d["beta_Exp_down"], d["beta_Exp_up"], s=5, alpha=0.16,
           color="steelblue", rasterized=True)
ax.plot([low, high], [low, high], color="crimson", lw=1.5, ls="--",
        label="45-degree line (symmetric)")
ax.set_xlim(low, high)
ax.set_ylim(low, high)
```

```
ax.set_xlabel(r"$\beta^{Exp}_{down}$")
ax.set_ylabel(r"$\beta^{Exp}_{up}$")
ax.set_title("Expense-rate beta on rate hikes vs. rate cuts")
ax.legend(fontsize=9)
ax.grid(alpha=0.3)
fig.tight_layout()
fig.savefig(OUT_FIG / "fig8_asym_scatter.png", bbox_inches="tight")
plt.close(fig)

# Figure 9 shows the full-sample distribution of bank-level hike-minus-cut
# differences. The vertical line marks equal pass-through.
diff = d["beta_Exp_up"] - d["beta_Exp_down"]
fig, ax = plt.subplots(figsize=(6.2, 4.2))
ax.hist(diff, bins=60, color="steelblue", alpha=0.82, edgecolor="white")
ax.axvline(0, color="crimson", lw=1.5, ls="--", label="Equal pass-through")
ax.axvline(diff.mean(), color="black", lw=1.2, ls=":", label=f"Mean = {diff.mean():.3f}")
ax.set_xlabel(r"$\beta^{Exp}_{up} - \beta^{Exp}_{down}$")
ax.set_ylabel("Banks")
ax.set_title("Distribution of bank-level pass-through differences")
ax.set_xlim(diff.quantile(0.005), diff.quantile(0.995))
ax.legend(fontsize=9)
ax.grid(axis="y", alpha=0.25)
fig.tight_layout()
fig.savefig(OUT_FIG / "fig9_asym_diff.png", bbox_inches="tight")
plt.close(fig)
```

Listing 7: `06_make_report.py` — Compile this report and append the code listings (this PDF).

```
"""
Bank Interest-Rate Sensitivity - Script 06
Build the LaTeX report (English), append the code listings, and compile to PDF.

All formulas are written in LaTeX source so they render and can be copied as
LaTeX. Sections are numbered in the order they appear. Variable names drop the
_w suffix in the text and tables (the rates used for estimation are winsorized
at 0.5/99.5 within quarter; this is stated in the table notes).

Run from the project root: python code/06_make_report.py
Outputs:
  output/bank_interest_rate_sensitivity.tex
  output/bank_interest_rate_sensitivity.pdf
"""
import subprocess
import sys
import re
from pathlib import Path

import pandas as pd
from scipy.stats import norm

ROOT = Path(__file__).resolve().parents[1]
PROC = ROOT / "data_processed"
OUT_TAB = ROOT / "output" / "tables"
OUT_FIG = ROOT / "output" / "figures"
OUT = ROOT / "output"
OUT.mkdir(parents=True, exist_ok=True)

```

```
FIG = Path("output") / "figures"  # relative paths keep the report portable
CODE = Path("code")               # scripts appended to the PDF as a code appendix
TAB = OUT_TAB

# ---------------------------------------------------------------------- helpers
def star_mark(t):
    """Two-sided asymptotic significance stars based on the reported statistic."""
    p = 2 * norm.sf(abs(float(t)))
    return "***" if p < 0.01 else "**" if p < 0.05 else "*" if p < 0.10 else ""


def coef_t(coef, t, digits=3):
    """Format a coefficient with significance stars and its statistic."""
    stars = star_mark(t)
    marker = f"$^{{{stars}}}$" if stars else ""
    return f"{float(coef):.{digits}f}{marker} ({float(t):.2f})"


def fmt_cell(s):
    """Add significance stars to regression cells formatted as coef (t-stat)."""
    if pd.isna(s):
        return ""
    value = str(s)
    match = re.fullmatch(r"\s*([+-]?\d+(?:\.\d+)?)\s*\(([+-]?\d+(?:\.\d+)?)\)\s*", value)
    if not match:
        return value
    coef, t = map(float, match.groups())
    stars = star_mark(t)
    marker = f"$^{{{stars}}}$" if stars else ""
    return f"{coef:.4f}{marker} ({t:.2f})"


def esc(s):
    """Escape LaTeX special characters in text cells (%, $, _)."""
    s = str(s)
    return (s.replace("\\", r"\textbackslash{}")
             .replace("%", r"\%")
             .replace("$", r"\$")
             .replace("_", r"\_"))

# ---------------------------------------------------------------------- data
t1 = pd.read_csv(f"{TAB}/table1_descriptives.csv")
t2 = pd.read_csv(f"{TAB}/beta_summary.csv")
t3 = pd.read_csv(f"{TAB}/table2_cross_section.csv")
t4 = pd.read_csv(f"{TAB}/table4_stock_regressions.csv")
t5 = pd.read_csv(f"{TAB}/table5_asymmetry.csv")

# Table 3 -> readable layout: Spec | Intercept | Slope | N | Adj R2 | Interaction
t3v = pd.DataFrame({
    "Specification": t3["Specification"],
    "Intercept": [coef_t(a, t) for a, t in zip(t3["Intercept"], t3["Intercept_t"])],
    "Slope": [coef_t(a, t) for a, t in zip(t3["Slope"], t3["Slope_t"])],
    "N": t3["N"].astype(int),
    "Adj R2": t3["Adj_R2"].round(3),
    "Interaction": [("" if pd.isna(a) else coef_t(a, t))
                    for a, t in zip(t3["Int_coef"], t3["Int_t"])],
})

L = []  # lines of the .tex file
```

```
def w(s=""):
    L.append(s)

# ============================================================== preamble
w(r"""\documentclass[11pt]{article}
\usepackage[margin=1in]{geometry}
\usepackage{amsmath,amssymb}
\usepackage{graphicx}
\usepackage{url}
\usepackage{booktabs}
\usepackage{array}
\usepackage{caption}
\usepackage{float}
\usepackage[T1]{fontenc}
\usepackage{lmodern}
\captionsetup{font=small,labelfont=bf}
\setlength{\parskip}{2pt}
\newcommand{\betai}{\hat{\beta}_{i}}
\usepackage{listings}
\usepackage{xcolor}
\definecolor{codebg}{rgb}{0.975,0.975,0.975}
\definecolor{codekw}{rgb}{0.10,0.25,0.55}
\definecolor{codecm}{rgb}{0.20,0.45,0.35}
\definecolor{codestr}{rgb}{0.45,0.30,0.10}
\lstdefinestyle{py}{
  language=Python,
  basicstyle=\ttfamily\footnotesize\linespread{1.0}\selectfont,
  keywordstyle=\color{codekw}\bfseries,
  commentstyle=\color{codecm}\itshape,
  stringstyle=\color{codestr},
  numberstyle=\tiny\color{gray},
  numbers=left, numbersep=6pt, stepnumber=1,
  breaklines=true, breakatwhitespace=false,
  showstringspaces=false, tabsize=4,
  frame=single, rulecolor=\color{gray!50},
  framexleftmargin=12pt, framextopmargin=4pt, framexbottommargin=4pt,
  backgroundcolor=\color{codebg},
  aboveskip=8pt, belowskip=14pt,
  upquote=true, columns=fullflexible, keepspaces=true
}
\lstset{style=py}
\begin{document}

\begin{center}
{\LARGE\bfseries Bank Interest-Rate Sensitivity\\[4pt]
\large Reproducible Empirical Analysis of U.S. Commercial Banks, 1984--2017}\\[10pt]
Cunhan Guo
\end{center}

\noindent\textbf{Reproducibility.} The scripts in \texttt{code/} read the four
raw files under \texttt{data/} and write processed data, tables, and figures
under \texttt{data\_processed/} and \texttt{output/}. Run scripts 01 through
06 in order from the project root. The bank-quarter panel spans
1984Q1--2017Q4.
""")

# ============================================================ 1. Data audit
w(r"""\section{Data Audit and Descriptive Evidence}
""")
```

```
148
149 w(r"""\subsection{Data and data-preparation choices}
150 Four files are provided. \path{callreports_1984_2017.dta} holds
151 1,114,194 bank-quarter rows for 17,804 distinct commercial-bank identifiers,
152 calendar quarter, quarterly average assets, total interest income, total
153 interest expense, and net income. After excluding rows with missing flows or
154 non-positive assets, 17,743 banks remain with at least one usable quarter;
155 61 raw identifiers have no usable observations. \texttt{FedFunds.csv} is the monthly
156 effective federal funds rate; I take the last month of each quarter as
157 $f_t$. \path{Bank_and_CRSP_Linking_Table.csv} and
158 \path{CRSP_Monthly_Stock_File.csv} are used in Section~\ref{sec:stocks}.
159
160 The three income/expense/net-income items are year-to-date (YTD) cumulative
161 flows, not single-quarter flows. I detected this because ROA built from raw
162 net income had a seasonal sawtooth: Q4 values ran systematically above Q1 for
163 almost every bank. I verified the cumulation directly: in complete bank-years,
164 the reconstructed quarterly flows sum exactly to the reported Q4 YTD value.
165 Single-quarter flows are recovered by within-year
166 differencing (Q1 flow = Q1 YTD; quarters 2--4: flow $=$ YTD$_t$ - YTD$_{t-1}$).
167 One timing guard: I difference only across adjacent calendar quarters (using
168 the year and quarter fields). A bank that skips a quarter gets a missing flow for that quarter
169 rather than a two-quarter change mislabeled as one quarter. Skipped quarters
170 are rare but concentrated among short-lived banks.
171
172 Missing values: 5,676 of 1,114,194 rows are dropped for missing flows or
173 non-positive assets (0.51\%), leaving 1,108,518 bank-quarters. Missingness is
174 highest in the mid-1980s and around 2005--06 and is near zero after 2010
175 (Figure~\ref{fig:miss}). Extreme observations: the three annualized rates are
176 winsorized at the 0.5/99.5 percentiles within each calendar quarter before
177 beta estimation, so a single merger or write-down quarter does not drive one
178 bank's time-series regression. The estimated betas are not trimmed further.
179 """)
180
181 w(r"""Fixed effects follow the variation used in each regression. The bank-level
182 time-series models include bank-specific intercepts; quarter effects would
183 absorb the common federal-funds-rate changes used to estimate the betas. The
184 cross-sections have one observation per bank, so bank fixed effects cannot be
185 estimated alongside the bank-level beta outcome; I use HC1 standard errors.
186 In the company-month stock-return regressions, company fixed effects would
187 change the question from between-company associations to within-company
188 changes. Month effects would absorb the common market return, so I include the
189 CRSP market return and cluster standard errors by company.
190 """)
191
192 w(r"""\begin{table}[H]\centering\small
193 \caption{Descriptive statistics, bank-quarter panel, 1984--2017
194 (N~=~1,108,518 bank-quarters, 17,743 banks). Assets are reported by the Call
195 Report in \$ thousands and converted to \$ millions. Rates are annualized
196 percent. Rates are winsorized at 0.5/99.5 within quarter; the \_w suffix is
197 dropped in the text and tables.}\label{tab:t1}
198 \begin{tabular}{lcccccc}\toprule
199 Variable & N & Mean & SD & p1 & Median & p99\\
200 \midrule""")
201 for _, r in t1.iterrows():
202     vname = esc(r["Variable"])
203     w(" & ".join([vname, str(int(r["N"])),
204                   f"{r['Mean']:.3f}", f"{r['SD']:.3f}",
205                   f"{r['p1']:.3f}", f"{r['Median']:.3f}", f"{r['p99']:.3f}"]) + r"\\")
206 w(r"""\bottomrule
```

```
\end{tabular}
\end{table}
""")

w(r"""The median bank is small (median average assets \$66.1m versus a mean of
\$757m), so a few hundred large banks account for most industry assets.
Median IncRate is 7.54\%, median ExpRate 3.34\%, and median ROA 1.04\%, with
the average federal funds rate over the sample at 4.74\%. Table~\ref{tab:t1}
shows the full distribution.
""")

w(r"""\begin{figure}[H]\centering
\includegraphics[width=0.72\textwidth]{""" + f"{FIG}/fig1_industry_over_time.png" + r"""}
\caption{Banking industry over time: number of reporting banks (left), total
assets (center), median bank assets (right).}\label{fig:industry}
\end{figure}
""")

w(r"""\begin{figure}[H]\centering
\includegraphics[width=0.62\textwidth]{""" + f"{FIG}/fig2_missing_values.png" + r"""}
\caption{Share of bank-quarters with missing income/expense/net-income items by
year, computed on the raw panel before dropping rows.}\label{fig:miss}
\end{figure}
""")

w(r"""\begin{figure}[H]\centering
\includegraphics[width=0.8\textwidth]{""" + f"{FIG}/fig3_rate_trends.png" + r"""}
\caption{Annual median IncRate, ExpRate, net interest margin (IncRate minus
ExpRate), and median ROA, 1984--2017. The margin is remarkably stable near
4 percentage points; ROA dips in 1987, 1991, and 2008--09.}\label{fig:trends}
\end{figure}
""")

w(r"""\subsection{Main patterns}
Three patterns stand out. First, the industry consolidated sharply: reporting
banks fell from about 13,500 in 1984 to about 4,500 by 2016, while total assets
grew more than sixfold (Figure~\ref{fig:industry}). Second, the
size distribution stayed extremely skewed (mean assets \$757m, median \$66m).
Third, the net interest margin hovered near 4 percentage points for the whole
sample, while median ROA moved with the cycle (Figure~\ref{fig:trends}).

Two data limitations matter for what follows. The panel is unbalanced and exit
is correlated with size, so time-series aggregates mix entry, exit, and
organic change. And the YTD reporting convention, if left untreated, would put
a mechanical seasonal component into every rate and every beta; the
reconstruction in the previous subsection is the main guard against that.
""")
# ============================================================ 2. Bank-level betas
w(r"""\section{Bank-Level Interest Income and Expense Betas}
""")
w(r"""For bank $i$ in quarter $t$ I construct annualized rates in percentage
points:
""")
w(r"""\begin{equation}
\begin{aligned}
\text{IncRate}_{i,t}&=400\times\frac{\text{interest income}_{i,t}}{\text{average assets}_{i,t}},\\
\text{ExpRate}_{i,t}&=400\times\frac{\text{interest expense}_{i,t}}{\text{average assets}_{i,t}}.
\end{aligned}
\end{equation}
```

```
""")
w(r"""using the reconstructed single-quarter flows. The federal funds rate
$f_t$ is the last monthly value of quarter $t$. First differences are
""")
w(r"""\begin{equation}
\Delta y_{i,t}=y_{i,t}-y_{i,t-1},\qquad \Delta f_t=f_t-f_{t-1},
\end{equation}
""")
w(r"""and, for each bank and each outcome $y\in\{\text{IncRate},\text{ExpRate},
\text{ROA}\}$, I estimate
""")
w(r"""\begin{equation}
\Delta y_{i,t}=a_i^y+\sum_{\tau=0}^{3}b_{i,\tau}^{y}\,\Delta f_{t-\tau}+u_{i,t}^y,
\end{equation}
""")
w(r"""with the four coefficients allowing the response to build over one year.
The bank's beta is the sum of the four estimated coefficients,
""")
w(r"""\begin{equation}
\widehat{\beta}_i^y=\sum_{\tau=0}^{3}\widehat{b}_{i,\tau}^{y}.
\end{equation}
""")
w(r"""I require at least 20 usable first-difference observations per bank
(about five years). This gives 13,624 eligible banks of 17,743 (77\%); most
excluded banks are short-lived or de novo. Median one-year betas are 0.358 for
income and 0.353 for expense. A one-percentage-point rise in the federal funds
rate is associated, within a year, with about a 0.36-point rise in the income
rate and a 0.35-point rise in the expense rate. The ROA beta distribution is
centered near zero (median -0.012, SD 0.70), so profitability is only loosely
tied to rate movements on average (Table~\ref{tab:t2}, Figure~\ref{fig:betadist}).
""")

w(r"""\begin{table}[H]\centering\small
\caption{Distribution of bank-level estimated betas (13,624 banks). Rates are
winsorized at 0.5/99.5 within quarter; the \_w suffix is dropped in the text
and tables. Betas are not trimmed further.}\label{tab:t2}
\begin{tabular}{lccc}\toprule
Statistic & Beta\_Inc & Beta\_Exp & Beta\_ROA\\
\midrule""")
for _, r in t2.iterrows():
    st = str(r["Statistic"])
    if st == "N banks":
        w("N banks & " + f"{int(r['Beta_Inc'])} & {int(r['Beta_Exp'])} & {int(r['Beta_ROA'])}" + r"
    \\")
    else:
        w(" & ".join([st, f"{r['Beta_Inc']:.3f}", f"{r['Beta_Exp']:.3f}", f"{r['Beta_ROA']:.3f}"])
    + r"\\")
w(r"""\bottomrule
\end{tabular}
\end{table}
""")

w(r"""\begin{figure}[H]\centering
\includegraphics[width=0.85\textwidth]{""" + f"{FIG}/fig4_beta_distributions.png" + r"""}
\caption{Histograms of bank-level income, expense, and ROA betas. The ROA
distribution is much wider than the two rate betas.}\label{fig:betadist}
\end{figure}
""")

```

```
w(r"""The cleaned bank-level dataset is submitted as
\texttt{data\_processed/bank\_betas.csv}: one row per bank (17,743 rows) with
idrssd, bank name, first/last quarter, mean assets, the three betas
(beta\_Inc, beta\_Exp, beta\_ROA), observation counts, R$^2$, and eligibility
flags. Banks below the 20-observation minimum keep a row with missing betas,
so the file contains every bank in the panel. Size in that file is each
bank's time-series average of total assets over its usable sample
(mean\_asset), the natural one-row-per-bank analogue of the quarterly asset
measure.
""")

# ================================================== 3. Income vs expense
w(r"""\section{Relationship between Income and Expense Sensitivities}
""")
w(r"""I regress income beta on expense beta by OLS:
""")
w(r"""\begin{equation}
\widehat{\beta}_i^{\text{Inc}}=\alpha+\gamma\,\widehat{\beta}_i^{\text{Exp}}+\varepsilon_i.
\end{equation}
""")
w(r"""I use HC1 heteroskedasticity-robust standard errors for the
cross-sectional regressions. The estimates are based on one observation per
bank; the table reports coefficients, HC1 t-statistics, sample sizes, and
adjusted R-squared values.
""")

w(r"""\begin{table}[H]\centering\small
\caption{Cross-sectional regressions of income beta on expense beta (Panel A)
and ROA beta on expense beta (Panel B), one observation per bank. HC1
t-statistics in parentheses. Size terciles are defined by each bank's
time-series average total assets (mean\_asset), not within-quarter assets.
For interaction rows, `Slope' is the expense-beta coefficient at average log
assets; the level term in log assets is not shown. Stars denote two-sided
significance at 10\%, 5\%, and 1\% (*, **, ***).}\label{tab:t3}
\begin{tabular}{llllll}\toprule
Specification & Intercept & Slope & N & Adj R2 & Interaction\\
\midrule""")
for _, r in t3v.iterrows():
    w(" & ".join([esc(r["Specification"]), str(r["Intercept"]), str(r["Slope"]),
                  str(r["N"]), str(r["Adj R2"]), str(r["Interaction"])]) + r"\\")
w(r"""\bottomrule
\end{tabular}
\end{table}
""")

w(r"""The estimated slope is positive and below one (Table~\ref{tab:t3},
Panel A): a 0.1 higher expense beta corresponds to about a 0.072 higher income
beta. This is consistent with asset and liability repricing moving together,
while income rates respond less than one-for-one on average. The positive
intercept indicates that banks with low expense sensitivity can still have
positive income sensitivity, as would be expected when some assets reprice
with short-term rates.

The relationship is positive in each size tercile, with a somewhat larger
slope among medium and large banks than among small banks. The interaction
between expense beta and demeaned log assets is not statistically significant;
the data do not show a clear linear size gradient (Figure~\ref{fig:incexp}).
""")

```

```
w(r"""\begin{figure}[H]\centering
\includegraphics[width=0.62\textwidth]{""" + f"{FIG}/fig5_inc_vs_exp.png" + r"""}
\caption{Binned scatter of income beta against expense beta (20
equal-frequency bins) with the OLS line. All eligible banks are included.}
\label{fig:incexp}
\end{figure}
""")

# ================================================== 4. Profitability
w(r"""\section{Interest Sensitivity of Profitability}
""")
w(r"""ROA is net income divided by average assets, annualized:
""")
w(r"""\begin{equation}
\text{ROA}_{i,t}=400\times\frac{\text{quarterly net income}_{i,t}}
{\text{quarterly average assets}_{i,t}},
\end{equation}
""")
w(r"""and the ROA beta comes from the same distributed-lag regression. Regressing
beta\_ROA on beta\_Exp gives a positive slope (HC1 t = 2.03), with adjusted
R$^2$ = 0.020 (Table~\ref{tab:t3}, Panel B). Expense beta explains only a
small share of cross-bank variation in ROA beta. The negative intercept is
hard to interpret mechanically because very few banks have exactly zero
expense sensitivity.
""")
w(r"""\begin{figure}[H]\centering
\includegraphics[width=0.62\textwidth]{""" + f"{FIG}/fig6_roa_vs_exp.png" + r"""}
\caption{Binned scatter of ROA beta against expense beta with the OLS line.
The positive slope is driven mostly by the upper bins; the fit is weak
everywhere (R$^2$ = 0.02).}\label{fig:roaexp}
\end{figure}
""")
w(r"""Size heterogeneity: the estimated slopes are positive across size groups,
but none is statistically distinguishable from zero at conventional levels.
The interaction with demeaned log assets is also insignificant
(Figure~\ref{fig:roaexp}). The overall message of this section is that profitability
sensitivity is weakly related to expense sensitivity, with most cross-bank
variation left unexplained.
""")

# ================================================== 5. Publicly traded banks
w(r"""\section{Publicly Traded Banks}\label{sec:stocks}
""")
w(r"""\subsection{Linking banks to stocks}
I map the bank panel to CRSP with \texttt{Bank\_and\_CRSP\_Linking\_Table.csv},
which links RSSD identifiers (bank idrssd, holding-company rssd9001) to
PERMCO. Matching is on the bank's own idrssd first; banks with no idrssd match
are then tried through the holding company's bhc\_id. I chose this order after
checking that a naive concat of the two match types could put the same bank
under two different PERMCOs and double-count its assets and flows in the
company aggregates. Each link is restricted to its validity window
[dt\_start, dt\_end]. This yields 124,929 bank-quarter links covering 5,163
banks and 1,062 listed companies (PERMCO). No bank-quarter has multiple
PERMCOs among the valid links.

Where several banks map to one company, I aggregate to the company level:
average assets, total assets, and the three quarterly flows are summed
(stocks and flows both aggregate linearly), and the rates are recomputed from
the sums. These records include only banks matched to a listed company, so
```

```
company assets and flows may be understated when subsidiaries are unmatched;
ambiguous links are excluded. Intra-company transfers are not netted out, and
partly owned subsidiaries are counted at 100\% rather than by ownership share.
Call Report data do not let me separate these components, so the resulting
consolidated rates may be biased.
""")
w(r"""\subsection{Total-return index}
CRSP lists multiple share classes (PERMNO) for the same company-month. Before
building any series I collapsed them using prior-month market-cap weights, so
a company's monthly return is the value-weighted average of its share classes.
The CRSP file contains 453,756 PERMCO-month duplicates from multiple share classes. Monthly returns
     use
MthRet, which includes dividends, so the compounded series is a total-return
index (2000 = 100).
""")
w(r"""\begin{figure}[H]\centering
\includegraphics[width=0.72\textwidth]{""" + f"{FIG}/fig7_bank_stock_index.png" + r"""}
\caption{Total return index of publicly traded banks (2000 = 100, dividends
included), equal-weighted and value-weighted, 2000--2017. The value-weighted
index lost about 60\% peak-to-trough in 2008--09; the equal-weighted index
fell far less, so most crisis losses were concentrated in the big
banks.}\label{fig:index}
\end{figure}
""")
w(r"""Figure~\ref{fig:index} shows that the value-weighted index fell much more
during 2008--09 and recovered more slowly than the equal-weighted index.
Returns among the largest listed bank companies drove much of the
value-weighted series' decline.
""")
w(r"""\subsection{Bank characteristics and stock returns}
To avoid look-ahead bias, each quarter's Call Report is treated as public two
months after quarter end (reports are filed with a lag); each monthly return
is matched to the most recent report whose public date is no later than that
month. The company-level expense beta is not used in return regressions
because a full-sample beta would embed post-return information; return tests
use only lagged accounting levels. After aggregation, company-level IncRate,
ExpRate, and ROA are recomputed from summed flows and assets, then winsorized
at the 0.5/99.5 percentiles within quarter. This applies the same percentile
cutoffs as the bank-level rates while defining outliers across linked
companies.
""")
w(r"""\begin{table}[H]\centering\small
\caption{Monthly stock returns of listed banks on lagged company
characteristics, 2000--2017. Coefficient (t-statistic clustered by listed
company). vwretd is the CRSP value-weighted market return; company rates are
winsorized within quarter. Stars denote two-sided significance at 10\%, 5\%,
and 1\% (*, **, ***).}\label{tab:t4}
\begin{tabular}{lccc}\toprule
Variable & (1) Market & (2) +Size+ROA & (3) +Rates\\
\midrule""")
for _, r in t4.iterrows():
    w(" & ".join([esc(r["Variable"])] + [fmt_cell(r[c]) for c in t4.columns[1:]]) + r"\\")
w(r"""\bottomrule
\end{tabular}
\end{table}
""")
w(r"""The market-return coefficient is positive and precisely estimated. Conditional
on the market return, larger company-level assets are associated with slightly
lower subsequent monthly returns. In the rate specification, income rate
```

```
enters positively and expense rate negatively, consistent with a positive
association between net interest margins and subsequent returns
(Table~\ref{tab:t4}). All three specifications use the same 77,720
company-month observations.
""")

# ================================================== 6. Asymmetric pass-through
w(r"""\section{Is the Pass-Through of Policy Rates to Bank Funding Costs Symmetric?}
""")
w(r"""\subsection{Empirical approach}
Section 2 estimated a single expense beta, implicitly assuming that funding costs
respond symmetrically to rate increases and decreases. Deposit rates are often
described as sticky downward: banks reprice deposits quickly when the Fed
raises, but cut them slowly when the Fed lowers. If so, a cut in the policy
rate should pass through to funding costs less than a hike of the same size,
widening margins during easing cycles. I test this directly by splitting each
quarterly change in the federal funds rate into an increase and a decrease
part,
""")
w(r"""\begin{equation}
\Delta f^{+}_t=\max(\Delta f_t,0),\qquad
\Delta f^{-}_t=\min(\Delta f_t,0),
\end{equation}
""")
w(r"""and estimating, for each bank,
""")
w(r"""\begin{equation}
\Delta \text{ExpRate}_{i,t}=a_i+\sum_{\tau=0}^{3}b^{\text{up}}_{i,\tau}\,\Delta f^{+}_{t-\tau}
+\sum_{\tau=0}^{3}b^{\text{down}}_{i,\tau}\,\Delta f^{-}_{t-\tau}+u_{i,t},
\end{equation}
""")
w(r"""with the two cumulative betas
""")
w(r"""\begin{equation}
\beta^{\text{up}}_{i}=\sum_{\tau=0}^{3}\widehat{b}^{\text{up}}_{i,\tau},\qquad
\beta^{\text{down}}_{i}=\sum_{\tau=0}^{3}\widehat{b}^{\text{down}}_{i,\tau}.
\end{equation}
""")
w(r"""The identification compares how a bank's expense rate moves in quarters
following rate hikes versus quarters following rate cuts, over the same
distributed-lag window as in Section 2. I require at least 28 usable observations
(seven years), leaving at least 19 residual degrees of freedom after estimating
eight lag coefficients and an intercept. The table reports the eligible sample
sizes. Size terciles use each bank's time-series mean of total assets, as in
Section 3. Estimated betas are not trimmed. All results are descriptive, not causal.
""")

w(r"""\begin{table}[H]\centering\scriptsize
\caption{Asymmetry in interest-expense pass-through. Mean difference is
$\beta_{up}-\beta_{down}$; positive values mean greater pass-through during
hikes. Share reports the fraction of banks with a positive difference.}
\label{tab:t5}
\setlength{\tabcolsep}{2.5pt}
\begin{tabular}{lrrrrrrrr}\toprule
Sample & N & Mean up & Mean down & Mean diff. & Median diff. & Share with diff $>0$ & Paired t & p-
    value\\
\midrule""")
for _, r in t5.iterrows():
    pval = "<0.001" if r["Paired p-value"] < 0.001 else f"{r['Paired p-value']:.3f}"
```

```
    w(" & ".join([esc(r["Sample"]), str(int(r["N banks"])),
                  f"{r['Mean beta_Exp_up']:.3f}", f"{r['Mean beta_Exp_down']:.3f}",
                  f"{r['Mean difference (up - down)']:+.3f}",
                  f"{r['Median difference (up - down)']:+.3f}",
                  f"{100*r['Share with diff > 0']:.1f}\%",
                  f"{r['Paired t-stat']:.2f}", pval]) + r"\\")
w(r"""\bottomrule
\end{tabular}
\end{table}
""")

w(r"""\begin{figure}[H]\centering
\includegraphics[width=0.62\textwidth]{""" + f"{FIG}/fig8_asym_scatter.png" + r"""}
\caption{Bank-level cumulative expense-rate betas for rate hikes and cuts. The dashed line shows equal pass-through; axes show the central 99\% of estimates. All eligible banks enter the summary table.}
\label{fig:asymscatter}
\end{figure}
""")

w(r"""\begin{figure}[H]\centering
\includegraphics[width=0.68\textwidth]{""" + f"{FIG}/fig9_asym_diff.png" + r"""}
\caption{Distribution of full-sample bank-level differences,
$\beta_{up}-\beta_{down}$. Axes show the central 99\% of estimates; all
banks are included in the table. The dashed line marks zero and the dotted
line marks the sample mean.}
\label{fig:asymdiff}
\end{figure}
""")

full = t5.iloc[0]
pre07 = t5.loc[t5["Sample"].str.startswith("Pre-2007")].iloc[0]
small = t5.loc[t5["Sample"].eq("Small")].iloc[0]
medium = t5.loc[t5["Sample"].eq("Medium")].iloc[0]
large = t5.loc[t5["Sample"].eq("Large")].iloc[0]
w(r"""\subsection{Results}
Table~\ref{tab:t5} reports the mean hike-minus-cut difference with its sign
shown explicitly. In the full sample it is """ + f"{full['Mean difference (up - down)']:+.3f}" + r""; the pre-2007 re-estimate is """ + f"{pre07['Mean difference (up - down)']:+.3f}" + r""". The sign reversal means the full-sample result does not show a stable asymmetry across periods. Figure~\ref{fig:asymscatter} compares the two betas directly, while Figure~\ref{fig:asymdiff} shows the distribution of their differences.

By size, mean differences are """ + f"{small['Mean difference (up - down)']:+.3f}" + r""", """ + f"{medium['Mean difference (up - down)']:+.3f}" + r""", and """ + f"{large['Mean difference (up - down)']:+.3f}" + r""" for small, medium, and large banks, respectively. These descriptive tercile comparisons do not establish a causal size pattern.

The sign has a direct interpretation: if $\beta_{up}>\beta_{down}$, expense rates pass through more during hikes, consistent with slower adjustment during cuts; if the two betas are equal, pass-through is symmetric; if $\beta_{down}>\beta_{up}$, pass-through is greater during cuts. Moreover, aggregate interest expense includes more than deposit costs, so this measure does not identify deposit pricing. Nor does the comparison by itself establish how net interest margins move, because asset yields also reprice. Hikes and cuts differ in number and size, bank-level betas are estimated with noise, and the results apply to banks meeting the 28-observation minimum. These estimates are descriptive, not causal.
""")

# ================================================== Conclusion
```

```
w(r"""\section{Conclusion}
Income and expense rates respond positively to federal funds rate changes,
and their bank-level betas move together. Profitability betas are only weakly
related to expense betas. In the stock-return regressions, the market return
is strongly associated with bank stock returns; accounting-rate associations
are modest. The hike-minus-cut difference is positive in the full sample
but negative in the pre-2007 re-estimation, so the evidence does not support a
stable asymmetry across periods. All findings are descriptive, not causal.
""")

# ================================================== code appendix
w(r"""\clearpage
\appendix
\section{Reproducible Code}
All seven scripts are released with this report. They read the four raw files
(Call Report, federal-funds series, bank--CRSP link table, CRSP monthly stock
file) from \texttt{data/}, write processed data to \texttt{data\_processed/},
and write tables and figures to \texttt{output/}; run them in the order shown.
The code is reproduced here verbatim, with line numbers.
""")

code_files = [
    ("01_data_prep.py", "Read the raw Call Report, undo YTD cumulation, annualize and winsorize the rates, and build the bank-quarter panel, descriptive table, and data-audit figures."),
    ("02_estimate_betas.py", "Estimate bank-level interest-income, interest-expense, and ROA betas by a distributed-lag regression on federal-funds changes."),
    ("02b_asym_betas.py", "Split federal-funds changes into positive and negative parts and estimate asymmetric expense betas for the full and pre-2007 samples."),
    ("03_cross_section.py", "Cross-sectional regressions of income and ROA betas on expense beta, with mean-assets terciles and a demeaned log-asset interaction."),
    ("04_stock_link.py", "Link banks to CRSP PERMCOs, aggregate share classes by prior-month market cap, build the total-return index, and run stock-return regressions."),
    ("05_q2_extension.py", "Summarize and plot the hike-versus-cut asymmetry, with paired tests and size terciles for the full and pre-2007 samples."),
    ("06_make_report.py", "Compile this report and append the code listings (this PDF)."),
]
for fn, desc in code_files:
    cap = r"\texttt{" + fn.replace("_", r"\_") + r"} \textemdash\ " + desc
    w(r"\lstinputlisting[language=Python,caption={" + cap + "},captionpos=t]{"
      + str(CODE / fn) + "}")

w(r"""\end{document}
""")

# ------------------------------------------------------------------ write
tex = "\n".join(L)
tex_path = OUT / "bank_interest_rate_sensitivity.tex"
tex_path.write_text(tex, encoding="utf-8")
print("wrote", tex_path)

# ------------------------------------------------------------------ compile
print("compiling with pdflatex ...")
res = subprocess.run(
    ["pdflatex", "-interaction=nonstopmode", "-halt-on-error",
     f"-output-directory={OUT}", str(tex_path)],
    capture_output=True, text=True, cwd=str(ROOT),
)
print(res.stdout[-2000:] if res.stdout else "")
print(res.stderr[-2000:] if res.stderr else "")
```

```
pdf = OUT / "bank_interest_rate_sensitivity.pdf"
if not pdf.exists():
    print("PDF NOT produced; see log above")
    sys.exit(1)
# second pass for labels/refs
subprocess.run(
    ["pdflatex", "-interaction=nonstopmode", "-halt-on-error",
     f"-output-directory={OUT}", str(tex_path)],
    capture_output=True, text=True, cwd=str(ROOT),
)
print("PDF:", pdf)
```